Graphic Era HILL UNIVERSITY

Established by an Act of the State Legislature of Uttarakhand (Adhiniyam Sankhya 12 of 2011)
University under section 2(f) of UGC Act, 1956

TITLE

RankRoom - Student Management and Competitive Programming Platform

Submitted in partial fulfilment of the requirements for the award of the degree

of

BACHELOR OF COMPUTER APPLICATIONS

by

LAKSHYA KUMAR

Under the supervision of

Dr. Chandrakala

SCHOOL OF COMPUTING,

GRAPHIC ERA HILL UNIVERSITY, CLEMENTOWN,

DEHRADUN-248002, UTTARAKHAND, INDIA

Graphic Era HILL UNIVERSITY

Established by an Act of the State Legislature of Uttarakhand (Adhiniyam Sankhya 12 of 2011)
University under section 2(f) of UGC Act, 1956

CANDIDATE'S DECLARATION

I hereby certify that the work which is being presented in the PROJECT REPORT entitled **RankRoom - Student Management and Competitive Programming Platform** in partial fulfilment of the requirements for the award of the Degree of Bachelor of Computer Applications and submitted in the School of Computing of the Graphic Era Hill University, Dehradun is an authentic record of my own work carried out under the supervision of **Dr. Chandrakala.**

Signature of the Candidate

This is to certify that the above statement made by the candidate is correct to the best of our knowledge.

Date: **Signature of Mentor**

Abstract

Digital transformation in higher education has improved access to learning resources, but most institutions still use fragmented systems for attendance, grading, assignments, coding practice, and event management. This fragmentation leads to duplicated effort, inconsistent records, delayed reporting, and weak student engagement. To address these gaps, this work proposes and implements RankRoom, a unified academic and skill-engagement platform that integrates academic workflows with competitive coding and analytics.

RankRoom is designed as a full-stack, role-driven system with Admin, Department Head, Teacher, and Student access scopes. The platform supports attendance sessions, grade publishing, assignment lifecycle management, coding problem practice, contest and hackathon participation, notifications, and leaderboard-driven motivation. The backend follows secure API design with role checks, scoped authorization, validation, and rate-limited execution flows. The data layer uses a relational schema for consistency and traceability, while real-time and asynchronous components support scalable submission processing and leaderboard updates.

The system combines institutional control with student-centric engagement. Routine academic indicators (attendance, internal marks, assignment completion) are connected to coding and competition outcomes to produce section-level and platform-level performance visibility. The integrated analytics module enables early identification of at-risk students and supports evidence-based interventions by faculty.

The developed system demonstrates that a single integrated platform can improve operational efficiency, transparency, and learner motivation more effectively than disconnected tools. RankRoom provides a practical model for institutions seeking to bridge academic administration and employability-focused skill development in one ecosystem.

Acknowledgement

I would like to express my sincere gratitude to myproject supervisor **Dr. Chandrakala**, for her continuous guidance, valuable suggestions, and unwavering support throughout the development of this project. Her expertise and mentorship played a vital role in helping us understand and implement the concepts involved in building this system.

I'm also deeply thankful to the faculty members of the School of Computing, Graphic Era Hill University for providing the necessary resources, academic environment, and encouragement required to successfully complete this project.

We would like to extend our appreciation to our team members for their cooperation, dedication, and effective teamwork during the entire development process. Their contributions were instrumental in successfully developing and integrating various modules of the system.

Finally, we express our heartfelt thanks to our family and friends for their constant encouragement, motivation, and support throughout the course of this work.

Signature of Candidate

Table of Contents

Abstract....2
Acknowledgement....3
CHAPTER 1: INTRODUCTION....6
1.1 Background....6
1.2 Problem Statement....8
1.3 Aim of the Project....10
1.4 Objectives....11
1.5 Scope....13
CHAPTER 2: LITERATURE REVIEW....15
2.1 Learning Management and Academic Systems....15
2.2 Role-Based Access Control in Education....16
2.3 Gamification for Student Engagement....18
2.4 Online Judge and Coding Assessment Systems....19
2.5 Distributed and Service-Oriented Architectures....22
2.6 Identified Research and Implementation Gaps....23
2.7 Justification for Proposed Work....25
CHAPTER 3: PROPOSED METHODS....27
3.1 System Design Philosophy....27
3.2 Architecture Method....28
3.3 Data Modeling Method....31
3.4 Access Control Method....34
3.5 Workflow Methods for Key Modules....35
3.6 Reliability and Security Methods....38
CHAPTER 4: System Design and Workflow....39
4.1 User Lifecycle Workflow....39
4.2 Academic Workflow....41
4.3 Assignment Workflow....42
4.4 Coding Problem Workflow....43
4.5 Contest Workflow....44
4.6 Hackathon Workflow....45
4.7 Notification and Reminder Workflow....46
4.8 Analytics Workflow....47
4.9 Architectural Diagrams....48
4.9.1 System Architecture....48

4.9.2 RBAC Model 49
4.9.3 Code Submission & Evaluation Pipeline 50
4.9.4 E.R. Diagram 51
4.9.5 Assignment and Contest Workflow 52
4.9.6 Module Interaction and Data Flow 53
CHAPTER 5: ANALYSIS AND DISCUSSION..... 54
5.1 Functional Analysis..... 54
5.2 Performance and Scalability Discussion..... 55
5.3 Engagement Discussion..... 56
5.4 Academic Monitoring Discussion..... 57
5.5 Security and Governance Discussion..... 58
5.6 Limitations..... 59
5.7 Overall Discussion..... 60
CHAPTER 6: CONCLUSION AND FUTURE WORK..... 61
6.1 Conclusion..... 61
6.2 Future Work..... 63
APPENDIX A..... 65
List of Abbreviations..... 65
LIST OF FIGURES..... 66
REFERENCES..... 71

CHAPTER 1: INTRODUCTION

1.1 Background

The rapid expansion of digital infrastructure across educational institutions has fundamentally altered the way academic processes are managed, monitored, and reported. Over the past two decades, universities and colleges worldwide have shifted from paper-based record-keeping to digital systems for attendance, grading, library management, and administrative communication. However, this transition has largely occurred in an uncoordinated manner, with individual departments and functional units independently adopting tools that best serve their narrow requirements without consideration for system-wide integration.

In most higher educational institutions today, the academic technology landscape is a patchwork of disconnected applications. Attendance may be recorded through a biometric device or a standalone web portal maintained by the administration office. Grades and internal assessments are often managed through spreadsheets or isolated faculty tools. Assignment submission may happen through a general-purpose file sharing service or an email thread. Announcements and notices flow through WhatsApp groups, email chains, or bulletin boards that are inconsistently monitored. Coding practice, competitive programming, and skill assessments happen entirely outside the institutional boundary, on external platforms such as LeetCode, HackerRank, or Codeforces, with no integration back into academic records.

This fragmented architecture creates a fundamental problem: the institution lacks a unified view of student progress. Faculty and administrators are forced to manually reconcile data from multiple sources when evaluating student performance, identifying struggling students, or generating reports for accreditation purposes. Students, meanwhile, experience a disjointed academic journey where their academic performance, skill development, and competition outcomes exist in separate silos with no unified identity or progression narrative.

Modern software engineering and enterprise systems design have long recognized that domain-driven, integrated platforms outperform collections of standalone tools when the underlying data is inherently relational. Academic data is precisely this kind of relational data — attendance affects grades, grades reflect assignment performance, coding outcomes

correlate with subject mastery, and all of these together paint a picture of student trajectory. Building a system that makes these relationships first-class citizens, rather than afterthoughts, is the core architectural motivation behind RankRoom.

Additionally, the rise of competency-based education and employability-focused curricula in engineering and computing programs has created a new expectation: institutions must not only track academic performance but also nurture and record skill development, competitive exposure, and extracurricular technical engagement. This expectation cannot be met by systems designed solely for administrative record-keeping.

RankRoom is designed at the intersection of these two needs — rigorous academic administration and meaningful skill engagement — and proposes a unified platform where both coexist under a consistent data model, role-based governance, and shared analytics infrastructure.

1.2 Problem Statement

The central problem this project addresses is the fragmentation of academic and skill-development workflows in higher educational institutions, particularly those offering computing and engineering programmes. This fragmentation manifests in several distinct and compounding ways.

Data Isolation: Each operational unit within an institution maintains its own records. Attendance systems store session and presence data independently. Examination cells maintain grade records in formats incompatible with other systems. Faculty maintain assignment records in personal spreadsheets. This isolation means that no single actor in the institution — not the student, not the teacher, not the department head — has access to a complete, up-to-date picture of student progress without manually aggregating data from multiple sources.

Delayed Reporting: Because data exists across disconnected systems, any form of cross-module reporting requires manual effort. Academic coordinators who wish to identify students at risk of failing due to poor attendance, incomplete assignments, and low assessment scores must perform this analysis by hand, often too late for effective intervention.

Weak Engagement Infrastructure: Students in computing programs are expected to develop skills beyond the curriculum — competitive programming, project work, hackathon participation, open-source contribution. Currently, these activities happen entirely outside institutional systems, providing no formal tracking, no academic credit linkage, and no visibility to faculty or administrators. This creates a gap between institutional accountability and actual student development.

Absence of Unified Identity: Students lack a unified academic and professional identity within the institution. Their coding achievements, contest rankings, project work, and academic scores exist in separate domains with no aggregation, making it impossible to generate a holistic student profile that could serve academic review, alumni tracking, or placement processes.

Inconsistent Motivation Frameworks: Without leaderboards, streaks, ranks, or visible progression indicators tied to academic and coding work, student engagement with optional learning activities is sporadic and declines over time.

RankRoom addresses each of these problems by providing a single, integrated, role-aware platform where academic workflows and skill-engagement activities share a unified data model, consistent authorization layer, and shared analytics infrastructure.

1.3 Aim of the Project

The aim of this project is to design, develop, and deploy RankRoom — a full-stack, role-driven academic and competitive programming platform that eliminates the fragmentation of institutional workflows by integrating academic administration, coding assessment, event management, and performance analytics within a single system boundary.

Specifically, the platform aims to:

- Provide a unified system where attendance, grades, assignments, problem solving, contest participation, and hackathon management coexist under one authentication layer and one data model.
- Enable role-appropriate visibility and governance so that students, teachers, class coordinators, department heads, and administrators each experience a workflow tailored to their responsibilities and authority.
- Deliver real-time and asynchronous infrastructure for processing code submissions, updating leaderboards, and dispatching notifications without degrading the responsiveness of primary user-facing interfaces.
- Produce analytics dashboards that correlate academic indicators with coding engagement indicators to provide actionable faculty insights and early at-risk identification.
- Create a gamified environment through leaderboards, ranks, streaks, and experience points that sustain student engagement with both academic requirements and optional skill-building activities.

1.4 Objectives

The following technical and functional objectives guide the development of RankRoom:

Role and Access Management: Build a hierarchical role system supporting Super Admin, Admin, Department Head, Class Coordinator, Teacher, and Student roles with scoped permissions at department and section levels. Each role must have clearly defined data visibility boundaries and operation privileges enforced at the API layer before any business logic executes.

Academic Module Implementation: Implement complete workflows for attendance session management, grade recording and publication, assignment lifecycle management including creation, targeting, submission, grading, and feedback, and subject-to-teacher and subject-to-section mapping. These modules must support the full operational cycle without requiring any external tool.

Coding and Competition Modules: Integrate a problem bank with structured test cases, editorial and hint support, and a fully functional online judge pipeline. Implement contest workflows with timed submission evaluation, standings computation using score and penalty logic, and result publication. Implement hackathon workflows including team formation, eligibility verification, registration management, and winner recording.

Scalable Submission Processing: Use asynchronous job queues to process code submissions against test suites without blocking user-facing API responses. Deliver verdicts, runtime data, and memory statistics through real-time channels after processing completes.

Analytics and At-Risk Detection: Compute aggregated analytics at the student and section level covering attendance percentage, grade trends, assignment completion rates, and coding activity metrics. Generate at-risk alerts when student data crosses configurable thresholds. Present this data through role-scoped dashboards.

Notifications and Reminders: Implement event-driven notification generation for academic deadlines, contest announcements, grade publications, and system events. Implement scheduled reminder jobs for near-deadline alerts. Support role, department, and section-level notification targeting.

Performance and Security: Apply rate limiting, input schema validation, secure middleware stacks, controlled file upload channels, and comprehensive activity logging across all API surfaces.

1.5 Scope

The scope of RankRoom encompasses the following functional and technical domains:

User and Role Lifecycle: Full user registration, authentication, profile creation, role assignment, and account management for all role levels from Super Admin to Student.

Institutional Structure: Department creation and management, section configuration, enrollment processing, and subject mapping with teacher assignment.

Attendance Management: Attendance session creation by teachers, student attendance recording, percentage calculation at subject and cumulative levels, and report generation for students and coordinators.

Grade Management: Grade entry for internal assessments, midterms, and finals; grade visibility controls; student performance summaries; and CGPA-adjacent tracking.

Assignment Lifecycle: Assignment creation with configurable due dates, scoring policies, and audience targeting; submission handling with file and code support; teacher grading and feedback publication; and integration with analytics modules.

Coding Problem Platform: Problem creation with difficulty classification, topic tagging, test case management, hint and editorial authoring, code execution for sample runs, judged submission evaluation, and verdict storage.

Contest Platform: Contest scheduling, problem set curation, student registration, real-time submission evaluation during contest windows, standing computation using score and time-penalty models, and result publication.

Hackathon Platform: Hackathon creation with eligibility criteria, individual and team registration, team validation, submission tracking, judging outcome recording, winner announcement, and profile reflection of achievements.

Leaderboard System: Global leaderboard tracking points from problem solving and contest performance; section-level leaderboard for local academic context; leaderboard recalculation on new accepted outcomes.

Analytics Module: Student-level dashboards showing attendance trends, grade progression, assignment completion, and coding activity. Section-level dashboards showing aggregate performance and at-risk student flags. Admin-level dashboards showing platform-wide activity.

Notification and Reminder Services: Event-triggered notifications, scheduled reminders, read/unread tracking, and multi-scope targeting.

Profile Enrichment: Student profile pages showing coding statistics, GitHub integration, LeetCode integration, skill tags, project listings, achievements, and contribution heatmaps.

CHAPTER 2: LITERATURE REVIEW

2.1 Learning Management and Academic Systems

Learning Management Systems have been the dominant paradigm for digital academic administration since the late 1990s. Platforms such as Moodle, Blackboard, and Canvas have achieved widespread adoption across universities globally, offering course content management, discussion forums, assignment submission portals, and basic grade recording. Academic research in educational technology has consistently validated the positive impact of LMS adoption on content accessibility and assignment submission rates.

However, a recurring finding in the literature is that LMS platforms are fundamentally course-centric rather than student-centric. They are designed to manage and deliver content for individual courses, not to aggregate and analyze student performance across their entire academic lifecycle. Grade data, for instance, is stored at the course level and rarely aggregated into holistic performance views without manual export and external analysis. Attendance, when supported at all, is typically a plugin feature with limited integration into the grade and analytics subsystems.

Studies in software systems engineering and enterprise architecture further reinforce the limitations of course-centric LMS design. Disconnected enterprise applications — even when individually functional — create data silos that increase operational overhead, reduce data reliability due to manual reconciliation errors, and impair decision-making quality for administrators and faculty who need cross-domain visibility. The total cost of maintaining multiple parallel systems — in terms of training, licensing, integration overhead, and data inconsistency resolution — consistently exceeds the cost of a well-designed integrated platform.

More recent academic work has begun exploring next-generation learning environments that go beyond traditional LMS capabilities by incorporating adaptive content delivery, competency tracking, social learning features, and skill marketplace integration. RankRoom aligns with this direction, specifically targeting the integration gap between academic administration and technical skill development that existing LMS platforms leave unaddressed.

2.2 Role-Based Access Control in Education

Role-Based Access Control (RBAC) is a widely studied and implemented authorization paradigm in which system permissions are associated with organizational roles rather than individual users. First formalized by NIST in the context of enterprise information systems, RBAC has been extensively applied to healthcare, government, finance, and increasingly to educational systems. The foundational NIST model defines four core components: Users, Roles, Permissions, and Sessions. Users are assigned to roles, roles are granted permissions, and users acquire permissions through the roles they hold during active sessions. This separation between user identity and permission sets is the central advantage of RBAC over discretionary access control models, where permissions must be managed individually for each user.

In educational contexts, the organizational hierarchy naturally maps onto an RBAC structure. Department heads must access data across all sections within their department but not across unrelated departments. Teachers must access attendance and grade records only for the sections and subjects they are assigned to. Students must see only their own records. Class coordinators need visibility into section-level aggregates without access to individual confidential records. This alignment between institutional hierarchy and role structure makes RBAC particularly well-suited to academic platforms.

Research on RBAC implementation in educational systems highlights that flat or coarsely-grained role models create either over-permission problems, where users can access data they should not, or under-permission problems, where legitimate workflows are blocked. A flat role model that grants all teachers the same permissions regardless of their assigned subjects, for example, would allow a teacher assigned to one department to view or modify grade records for a student in a different department. This represents both a data privacy failure and a governance failure, since the institution loses the ability to establish accountability for record modifications.

Fine-grained, hierarchical RBAC models that scope permissions not only to roles but also to organizational units — departments, sections, and enrolled cohorts — significantly reduce both categories of error. The NIST RBAC standard itself defines a hierarchical extension in which roles inherit permissions from parent roles, enabling the design of layered permission models where a department head role inherits the viewing permissions of the teacher role but

adds department-level administrative permissions on top. This hierarchical inheritance mirrors the supervisory relationships within educational institutions and reduces the configuration overhead of defining permissions for each role from scratch.

Additionally, literature on privacy compliance in educational systems, particularly in jurisdictions with student data protection regulations such as India's emerging data protection framework and analogues like FERPA in the United States, emphasizes that scope-bound access control is a prerequisite for compliance. Student academic records are personally identifiable information and are subject to legal protections that restrict who may access them and for what purposes. Section-level data scoping ensures that a teacher in one section cannot inadvertently view or modify records belonging to a section they are not assigned to, reducing both accidental and intentional data exposure.

The technical implementation of RBAC in web-based systems presents several architectural challenges. In monolithic systems, role checks can be embedded directly in business logic, but this approach leads to permission logic scattered throughout the codebase, making it difficult to audit and maintain. Modern system architectures prefer to centralize authorization in middleware layers that intercept every request before it reaches business logic, ensuring consistent enforcement regardless of which handler processes the request. This middleware-first approach also simplifies auditing, since all authorization decisions flow through a single code path that can be logged uniformly.

Attribute-Based Access Control (ABAC) is a generalization of RBAC that evaluates permissions based on arbitrary attributes of users, resources, and environmental conditions. While ABAC provides greater expressive power, its complexity makes it difficult to reason about, audit, and implement correctly in resource-constrained environments. Research comparing RBAC and ABAC implementations in enterprise systems generally finds that RBAC is preferable when the permission structure aligns naturally with a stable organizational hierarchy, as it does in educational institutions. ABAC is better suited to environments where access decisions depend on dynamic contextual conditions such as time of day, location, or data sensitivity labels that cannot be captured in role assignments alone.

RankRoom implements a multi-dimensional RBAC model where permission evaluation considers both role type and organizational scope — an approach validated by both security literature and practical institutional requirements. Each API request is evaluated against both

dimensions before any data access occurs. A teacher's request to record attendance is permitted only if the teacher's role type is TEACHER and the section specified in the request is among the sections assigned to that teacher in the database. A department head's request to view analytics is permitted only if the department specified belongs to the department head's assigned department. This two-dimensional evaluation creates a permission grid that is both precise and auditable.

The implementation further distinguishes between operation types — read, create, update, delete — and enforces different role requirements for each. A class coordinator may be permitted to read grade data for their section but not to modify it. A teacher may be permitted to create attendance sessions for assigned subjects but not to delete sessions created by another teacher. This operation-level granularity is consistent with the principle of least privilege, ensuring that each role holds exactly the permissions required to perform its defined responsibilities and no more.

2.3 Gamification for Student Engagement

Gamification — the application of game-design elements such as points, badges, leaderboards, streaks, levels, and challenges to non-game contexts — has been an active area of research in educational psychology and instructional design for over a decade. The theoretical foundations draw from Self-Determination Theory, which identifies competence, autonomy, and relatedness as the three core psychological needs whose satisfaction drives intrinsic motivation.

Empirical studies on gamification in educational settings report consistent positive effects on behavioral engagement — measured as participation rates, assignment completion rates, and platform interaction frequency — when gamification elements are tied to meaningful outcomes and visible progression. Crucially, the research distinguishes between surface-level gamification, where points and badges are awarded without connection to learning goals, and deep gamification, where progression indicators reflect genuine competency development.

RankRoom's gamification design reflects the findings of the deep gamification literature. Leaderboard positions are computed from problem-solving outcomes, contest standings, and academic performance metrics — all of which have direct educational relevance. Streak systems track consistent daily or weekly engagement with coding problems, providing

ongoing behavioral reinforcement. Section-level leaderboards create local competition within familiar social groups, which research identifies as more motivationally effective than global competition against unknown peers.

Furthermore, studies on competitive programming specifically find that contest participation produces strong motivation effects driven by the combination of clear objectives, immediate feedback, and social comparison. Embedding contest and hackathon participation within the same platform as academic records creates a unified motivation context where skill engagement and academic accountability reinforce each other rather than competing for student attention.

2.4 Online Judge and Coding Assessment Systems

Online judge systems — platforms that automatically evaluate code submissions against predefined test suites and return verdicts — have been a cornerstone of competitive programming infrastructure since the early 2000s. Systems such as Codeforces, SPOJ, and later platforms like LeetCode and HackerRank have demonstrated that automated evaluation at scale is both technically feasible and educationally effective for developing algorithmic problem-solving skills. The online judge model has since been adopted beyond competitive programming into technical interview preparation, university course assessment, and corporate hiring pipelines, demonstrating its versatility as an evaluation infrastructure.

The educational effectiveness of automated code evaluation is well-supported in the literature on formative assessment. Immediate feedback — receiving a verdict within seconds of submission — produces stronger learning outcomes than delayed feedback from manual grading, which may arrive days or weeks after a student completes a problem. The specificity of online judge verdicts also contributes to learning efficiency: a Time Limit Exceeded verdict directs a student to optimize their solution's runtime complexity, while a Wrong Answer verdict on a specific test case provides a concrete failure case for debugging. This specificity is qualitatively different from the generic score or percentage that manual grading typically produces.

The technical architecture of online judges has been extensively documented in systems literature. The core pipeline involves receiving a code submission, compiling it in a sandboxed environment, executing it against one or more test cases with resource limits

applied, comparing outputs against expected results, and returning a verdict — Accepted, Wrong Answer, Time Limit Exceeded, Memory Limit Exceeded, Runtime Error, or Compilation Error. Each stage of this pipeline presents distinct engineering challenges.

The compilation stage must handle multiple programming languages, each requiring different compiler or interpreter invocations. Modern online judges support dozens of languages, from compiled languages like C++ to interpreted languages like Python and JavaScript. The compilation environment must be isolated to prevent submitted code from interfering with the judge infrastructure. Language runtime environments have different memory footprints and startup times, which affects the resource limits that can realistically be applied and the throughput the system can sustain under concurrent submission load.

The execution stage is the most security-critical component of the online judge pipeline. Submitted code is untrusted and may attempt to read sensitive files from the host filesystem, make network connections to external servers, fork child processes to escape resource limits, or execute system calls that modify the host environment. Effective sandboxing requires restricting the set of system calls available to the submitted program, limiting filesystem access to a temporary directory, enforcing CPU time and wall-clock time limits separately to prevent timing attacks, and applying memory limits that prevent heap exhaustion on the host system. Modern judge implementations use Linux namespaces and cgroups for isolation, providing kernel-level resource control without the overhead of full virtual machine isolation.

The output comparison stage must handle edge cases that are easy to overlook but frequently cause incorrect verdicts in naive implementations. Trailing whitespace, Windows versus Unix line endings, and floating-point precision differences in expected outputs all require careful handling. Most online judges perform whitespace-normalized comparison by default and provide a special judge option for problems where exact output matching is inappropriate, such as problems with multiple valid answers or numerical answers where a tolerance must be applied.

Survey literature on online judge architectures highlights several recurring challenges beyond the core evaluation pipeline. Test case management — storage, versioning, and consistency — requires careful schema design to ensure that different submissions of the same problem are evaluated against identical test conditions. If test cases are modified after a problem is published, previously accepted submissions may produce different verdicts on the new test

cases, creating inconsistency in the historical record. Production judge systems typically version test case sets and record the version in effect at the time of each submission to maintain historical consistency.

Performance under peak load is a critical concern for contest-hosting judge systems. During a competitive programming contest, hundreds or thousands of participants may submit solutions simultaneously within the final minutes of the contest window. The submission queue can grow faster than it is drained if worker capacity is insufficient, causing evaluation latency to increase and potentially disadvantaging students whose submissions are evaluated later. Horizontal scaling of worker processes, combined with queue-based architecture that decouples submission acceptance from evaluation, is the standard approach for handling these burst load conditions.

A consistent finding in the literature is the integration gap between standalone online judge systems and institutional academic records. External platforms like LeetCode and HackerRank provide no mechanism for integrating a student's problem-solving history into their institutional academic record, denying faculty visibility into an important dimension of student development. Faculty who wish to assign competitive programming problems as coursework must track student completion manually, since there is no API or data export that connects external platform activity to the institution's grade management system. RankRoom addresses this gap by implementing an online judge pipeline native to the institutional platform, ensuring that coding outcomes are stored alongside academic records in the same data model and are immediately available to faculty analytics dashboards without any manual data transfer.

The Judge0 evaluation engine, which RankRoom integrates, is an open-source, language-agnostic code execution service that implements the sandboxed evaluation pipeline described above. Judge0 exposes a REST API for submission creation and result retrieval, supporting batch submission of multiple test cases in a single API call. This batch API is particularly important for RankRoom's contest use case, where a submission must be evaluated against dozens of hidden test cases and the aggregate result must be computed from the individual verdicts. Judge0's containerized architecture isolates each submission execution, preventing cross-submission interference even under concurrent load.

2.5 Distributed and Service-Oriented Architectures

Modern web-scale systems, including educational platforms serving large student populations, increasingly adopt architectural patterns borrowed from distributed systems engineering. Service-oriented architecture, microservices, and event-driven design patterns have been widely studied and applied in industry, with significant academic literature analyzing their tradeoffs.

For educational platforms, the most relevant distributed system considerations include asynchronous task processing for computationally intensive workloads, real-time communication for live status updates, horizontal scalability to handle simultaneous submission bursts during contests, and resilient middleware design that maintains user experience even when backend services experience degraded performance.

Research on background job processing frameworks highlights the advantages of queue-based architectures for submission evaluation pipelines. By placing submission jobs on a durable message queue and processing them through dedicated worker processes, the system decouples submission acceptance — which must be fast and reliable — from submission evaluation — which may take several seconds and is highly variable in duration. This decoupling improves perceived performance for users and allows the evaluation capacity to be scaled independently of the web API layer.

Real-time communication for verdict delivery and leaderboard updates has been addressed in the literature through WebSocket connections and server-sent events. WebSocket-based approaches, as implemented in frameworks like Socket.io, enable bidirectional persistent connections that allow the server to push verdict updates to connected clients the moment evaluation completes, eliminating the need for polling and improving the real-time feel of the submission experience.

2.6 Identified Research and Implementation Gaps

A synthesis of the literature across the five domains reviewed — learning management systems, role-based access control, gamification, online judge systems, and distributed architectures — reveals several consistent and compounding gaps that directly motivate the design of RankRoom.

The first and most fundamental gap is the absence of unified platforms that integrate both academic administration and coding engagement within a single data model and authentication boundary. This gap exists because the two domains have historically developed independently, served by different vendor ecosystems with different primary users. Learning management systems were designed for content delivery and course administration, with faculty and administrators as their primary users. Competitive programming platforms were designed for skill development and contest hosting, with students and contest organizers as their primary users. No widely available platform has successfully bridged both domains, leaving institutions to manage two separate ecosystems with incompatible data models and no shared identity layer.

The consequences of this first gap extend beyond operational inconvenience. When coding activity exists outside the institutional data model, it cannot be incorporated into academic analytics, cannot be referenced in at-risk detection algorithms, and cannot be used to correlate skill development with academic performance. An institution using a combination of Moodle for course management and Codeforces for competitive programming practice has no practical mechanism for identifying whether students who struggle academically also disengage from coding practice, or whether contest participation correlates with improved grades in algorithm-related subjects. This analytical blindness prevents evidence-based pedagogy from being applied to the coding skill domain.

The second gap is the lack of end-to-end scoped permissions that follow institutional hierarchy in existing educational platforms. Moodle's role system, for example, defines roles at the course level but does not natively model the department-section-subject hierarchy that characterizes most engineering institutions. Adapting Moodle's permission model to enforce department-level data isolation between teachers in different departments requires significant customization work that is not part of the standard installation. Most institutions that deploy

LMS platforms either accept coarser-grained permissions that create over-permission risks, or invest in customization work that is expensive to maintain across platform upgrades.

The third gap is the absence of integrated analytics that correlate academic indicators with technical engagement indicators within a single platform. The value of such correlations is well-established in the educational data mining literature, where models that combine behavioral engagement signals with academic performance measures consistently outperform models that use either signal type alone in predicting student outcomes. However, realizing this analytical value requires that both signal types — academic records and coding engagement data — be stored in the same data layer with a shared student identity, which is precisely what disconnected systems cannot provide.

The fourth gap is the lack of actionable at-risk detection within institutional platforms that operates on live data rather than periodic exports. Most at-risk identification systems in use at higher education institutions operate in batch mode, processing weekly or monthly data exports from multiple source systems. This batch processing introduces detection delays that reduce the window for effective intervention. A student who begins missing attendance sessions in the third week of a semester may not be flagged as at-risk until the fifth or sixth week, by which point multiple intervention opportunities have been missed. Real-time at-risk detection requires that the platform processing attendance data also be the platform that evaluates at-risk rules and delivers alerts to faculty, which is only possible in an integrated system.

The fifth gap, less prominently discussed in the literature but significant in practice, is the absence of student-facing unified identity and progression narratives in institutional platforms. Students who develop strong competitive programming skills, win hackathons, or achieve high LeetCode rankings have no mechanism for surfacing these achievements within their institutional academic profile. This disconnection means that faculty and placement officers who evaluate students for opportunities cannot easily discover coding achievements that are highly relevant to placement decisions but exist only on external platforms. An integrated platform that aggregates institutional academic records with external coding platform statistics produces a richer student profile that serves both motivation and placement purposes.

2.7 Justification for Proposed Work

RankRoom is justified by the convergence of a clearly documented institutional need — as evidenced by the fragmentation problem described in the problem statement and reinforced by the gaps identified in the literature review — and the current availability of mature, open-source technical components capable of addressing it without requiring proprietary infrastructure investment.

The institutional need is not merely a matter of convenience. The fragmentation of academic and skill-development workflows has measurable costs: faculty time spent on manual data reconciliation, delayed identification of at-risk students, reduced student engagement with optional skill-building activities due to the absence of institutional recognition, and the inability to generate holistic student performance reports for accreditation purposes. These costs are borne by every institution that operates disconnected systems, and they compound over time as the number of data sources increases and the institutional reliance on analytics grows.

The availability of mature open-source components is a critical enabling condition for RankRoom's feasibility. A decade ago, building a platform of this complexity would have required either proprietary database infrastructure, custom-built real-time communication components, or a large engineering team capable of implementing an online judge pipeline from scratch. The current ecosystem provides Supabase for managed PostgreSQL with row-level security and authentication, BullMQ for production-grade job queue management backed by Redis, Socket.io for battle-tested WebSocket communication, and Judge0 for open-source sandboxed code evaluation — all available under permissive licenses and with active maintenance communities. This combination makes it feasible for a small development team to build a platform of institutional-grade capability without the infrastructure investment that similar systems required in previous years.

RankRoom's contribution to the educational technology domain is architectural rather than algorithmic. The platform does not introduce novel machine learning methods, new programming language features, or innovative educational content. Its contribution is the validated integration architecture that demonstrates how academic administration and competitive skill engagement can coexist within a coherent, maintainable, and extensible platform using existing open-source components. This architectural contribution is directly

replicable by other institutions, making RankRoom a practical model rather than merely a research prototype.

The platform also addresses a specific deployment context — Indian engineering and computing institutions with explicit competency and skill development requirements embedded in their curriculum frameworks — where the gap between institutional administration systems and coding skill development platforms is particularly acute. The emphasis on coding competitions, hackathons, and coding practice as components of both formal curriculum and placement preparation in this context creates a strong institutional demand for the integration that RankRoom provides

CHAPTER 3: PROPOSED METHODS

3.1 System Design Philosophy

RankRoom is built on four foundational design principles that guide every architectural decision, technology selection, and implementation tradeoff.

Single Source of Truth: All academic and competitive data — attendance, grades, assignments, coding submissions, contest standings, leaderboard positions — resides in a single relational database schema. There are no parallel data stores for different modules, no synchronization requirements between separate systems, and no risk of data divergence. Every query, report, and analytics computation draws from the same underlying data, ensuring consistency across all system views.

Least-Privilege Authorization: Every API endpoint enforces authorization checks before executing any business logic. Authorization is two-dimensional: role-based checks determine whether a user's role type permits the requested operation, and scope-based checks determine whether the user's organizational assignments include the data being accessed. A teacher can only record attendance for sections they are assigned to. A department head can only view data within their department. This model is enforced at the middleware layer and cannot be bypassed by client-side manipulation.

Scalable Execution Through Asynchronous Processing: Computationally intensive operations — primarily code submission evaluation — are processed asynchronously through a job queue system. This ensures that user-facing API responses remain fast and reliable even under high concurrent submission load.

Continuous Feedback Through Dashboards and Notifications: The platform is designed to close feedback loops at every level. Students receive immediate verdict feedback after code evaluation. Faculty receive dashboard updates as attendance and grade data accumulates. Coordinators receive at-risk flags when student data crosses thresholds. Notification and reminder systems ensure that time-sensitive information — approaching deadlines, contest starts, grade publications — reaches the right users through the right channels without requiring manual communication.

3.2 Architecture Method

RankRoom follows a layered, service-oriented architecture organized into five distinct layers, each with clearly defined responsibilities and interfaces. The layered model ensures that each layer communicates only with adjacent layers, preventing the tangled inter-layer dependencies that characterize poorly structured applications and make them difficult to test, modify, and scale.

Presentation Layer: A Next.js application using the App Router paradigm provides server-side rendering for initial page loads and client-side navigation for subsequent interactions within the same session. Server-side rendering on initial load is important for two reasons: it reduces the time to first meaningful content for users on slower network connections, and it ensures that search engine crawlers receive fully rendered HTML for any public-facing content.

Role-specific dashboards, module views, and navigation structures are rendered based on the authenticated user's role and organizational scope, which are extracted from the JWT token present in the session. The Monaco Editor component — the same editor used in Visual Studio Code — is embedded within the coding problem interface, providing syntax highlighting, autocomplete, and error highlighting for the supported programming languages. This professional-grade editing experience reduces friction in the coding workflow and aligns with the tools students will use in professional contexts.

Service and API Layer: An Express.js application running on Node.js hosts the business logic and API surface. All API routes are protected by authentication middleware that validates JWT tokens and extracts role and scope information before any handler executes. The middleware stack processes each request through a defined sequence: rate limiting, body parsing, authentication, role verification, and request logging, before the request reaches the domain handler. This sequential middleware model ensures consistent treatment of all requests and provides clear insertion points for cross-cutting concerns.

Business logic modules are organized by domain — academic, coding, contest, hackathon, notification, analytics — with clear boundaries between them. Each domain module exports a router that is mounted under a dedicated path prefix, and a service layer that encapsulates the business logic and database interactions for that domain. This separation between routing and

service logic enables unit testing of business logic without requiring an HTTP server, and enables the service layer to be called from multiple entry points — HTTP handlers, queue workers, and scheduled jobs — without code duplication.

Data Layer: A PostgreSQL database hosted on Supabase provides the relational data store. PostgreSQL is chosen over alternative database systems for several reasons relevant to the academic platform use case. Its support for complex join queries with multiple aggregation levels, window functions for ranking computations, and JSON columns for flexible metadata storage makes it suitable for the heterogeneous data requirements of the platform. Its mature row-level security feature, offered through Supabase, provides an additional defense-in-depth layer beyond application-level authorization, ensuring that even direct database connections cannot access data beyond their authorized scope.

The schema is managed through Prisma ORM, which provides type-safe database queries, schema migration tooling, and relationship traversal. Prisma's type generation from the schema definition ensures that database query code is checked at TypeScript compilation time, catching schema-code mismatches before they reach production. The migration tooling maintains a versioned history of schema changes, enabling reproducible database setup across development, staging, and production environments.

Queue and Worker Layer: BullMQ, backed by Redis, provides the job queue infrastructure for asynchronous processing. The primary queue handles code submission evaluation jobs. Secondary queues handle scheduled reminder jobs, notification batching, and analytics recomputation tasks. Queue persistence in Redis ensures that jobs survive server restarts without loss. BullMQ's concurrency controls allow the number of simultaneously processed jobs to be configured per worker, preventing the worker from overwhelming the Judge0 evaluation service with more concurrent requests than it can handle efficiently.

Worker processes are deployed as separate Node.js processes from the API layer, enabling independent scaling. During periods of low submission volume, a single worker process is sufficient. During contest events when submission volume spikes, additional worker processes can be added without modifying or restarting the API layer. This elastic capacity model is one of the key scalability advantages of the queue-based architecture over inline evaluation approaches.

Real-Time Channel: Socket.io provides WebSocket-based real-time communication for verdict delivery, live leaderboard updates during contests, and notification delivery. The Socket.io server integrates with the Express.js application and maintains room-based subscriptions. When a student submits code, the client joins a room identified by the submission ID. When the worker completes evaluation and emits a verdict event to that room, Socket.io delivers the event to all connected clients in the room — in this case, exactly one client, the submitting student. For contest standings, all registered participants subscribe to a room identified by the contest ID, and standing update events are broadcast to all participants in the room whenever a submission is evaluated.

3.3 Data Modeling Method

The relational schema is organized around five conceptual clusters of entities, each representing a distinct functional domain while maintaining referential connections to entities in other clusters. The schema design follows the principle of normalization to third normal form, eliminating transitive dependencies that would cause update anomalies, while selectively denormalizing computed summary statistics into dedicated fields where the cost of recomputation on every read would be prohibitive.

Identity and Organizational Cluster: The User entity serves as the root identity anchor for all actors in the system. It stores authentication credentials in hashed form — the raw password is never stored — role assignments as an enumerated type, and account status flags. The design choice to store the role as a single enumerated field rather than through a user-to-role junction table reflects the assumption that each user holds exactly one primary role in the system. Users who need to perform operations across multiple roles — such as a faculty member who also participates as a student in professional development activities — are represented as separate user accounts with distinct email addresses and roles.

The Profile entity extends User with biographical, social, and professional information that is not required for authentication or authorization but enriches the user's identity within the platform. The extension relationship is implemented as a one-to-one foreign key from Profile to User rather than as additional columns on the User table, keeping the User table compact and authentication queries fast. The StudentProfile entity further specializes the profile for student users, capturing academic-specific attributes including enrollment year, section membership, and pre-computed academic summary statistics that are updated asynchronously by the analytics pipeline.

Department and Section entities form the organizational hierarchy. The Department entity is the top-level organizational unit, owned by an institution and supervised by a department head user. The Section entity represents an academic cohort within a department — a group of students who share the same subject curriculum for an academic year. Sections are the primary unit of enrollment, attendance tracking, and grade recording. The design decision to use Section as the enrollment unit rather than individual Subject allows students to be enrolled once in a section and automatically linked to all subjects offered within that section, reducing administrative overhead compared to subject-by-subject enrollment.

Academic Operations Cluster: The Subject entity models an academic course offered within a section, taught by a teacher, and characterized by a grading scheme that defines the weights of different assessment components. The subject-to-section and subject-to-teacher relationships are maintained through junction tables rather than direct foreign keys on the Subject entity, enabling many-to-many relationships where a subject may be offered in multiple sections or co-taught by multiple teachers.

The AttendanceSession entity records the metadata of each class meeting — the date, the subject, and the teacher who conducted the session. The AttendanceRecord entity records each student's status in each session with a three-value enumeration: Present, Absent, or Late. Late status is treated as present for attendance percentage computation by default, but can be configured to count as half a session at the institution level. This three-value model is more expressive than a binary present/absent model and supports institutions with formal late arrival policies.

The Grade entity stores assessment marks by component, with one record per student per subject per assessment component. The component-based storage model — rather than storing a single composite grade — preserves the raw component data for drill-down analytics and enables retrospective reweighting if a grading scheme is modified during the semester. A GradeConfiguration entity linked to the Subject stores the formula for computing composite scores from component marks, enabling different subjects to use different grading formulas within the same platform.

Coding Platform Cluster: The Problem entity is the central entity of the coding module. Its design must balance completeness — storing all information needed to present the problem to a student and evaluate submissions — with efficiency. The problem statement, which may be several paragraphs of rich text with embedded mathematical notation, is stored in a text column using Markdown format that is rendered client-side, avoiding the storage overhead of pre-rendered HTML. The constraints field, which specifies the input size and value ranges, is stored separately from the statement to enable structured filtering of problems by constraint characteristics.

The TestCase entity stores individual test cases linked to a Problem. The separation between sample test cases — which are visible to students as examples — and hidden test cases — which are used for evaluation — is implemented through a boolean is_sample flag rather than

through separate entities. This unified storage model simplifies test case management while preserving the visibility distinction through application-layer filtering. Each test case stores the input and expected output as text, with support for multi-line inputs through standard line-ending conventions.

The Submission entity is the most write-intensive entity in the schema. During a contest, hundreds of submissions may be created per minute. The entity stores the source code, language identifier, verdict, runtime in milliseconds, memory usage in kilobytes, the number of test cases passed, and processing timestamps. The source code is stored in the database rather than in object storage to keep verdict retrieval simple — a single query retrieves all submission metadata without a secondary object storage lookup. Source code sizes for typical competitive programming solutions are small enough that database storage is practical.

Competition Cluster: The Contest entity stores timed competition metadata. Its relationship to Problems is many-to-many through the ContestProblem junction entity, which adds problem-specific attributes like the point value assigned to the problem within the contest context. This design allows the same problem to appear in multiple contests with different point values without requiring problem duplication. The ContestStanding entity is updated after each accepted submission within the contest window and stores the cumulative score and time penalty for each participant, enabling efficient standings retrieval without real-time aggregation.

The Hackathon entity models a different form of competitive event with team participation, eligibility filtering, and project submission tracking. The HackathonTeam entity stores team metadata and is linked to the Hackathon through a foreign key. Individual team members are linked to HackathonTeam through a junction entity that records the member's role — captain or member — enabling team management operations like invitation acceptance and member removal to be implemented cleanly.

Engagement and Observability Cluster: The Leaderboard entity aggregates point totals from problem solving and contest performance into a single sortable score per user. Rather than computing leaderboard rankings on demand through a complex aggregation query, the Leaderboard entity stores pre-computed point totals that are updated incrementally when new accepted submissions or contest results are recorded. This pre-computation approach trades

some storage space for dramatically faster leaderboard retrieval, which is critical for the section-level and global leaderboards that are displayed on student dashboards.

The Notification entity models messages delivered to users about system events. The NotificationRecipient entity creates a per-user record for each notification, storing the read status independently for each recipient. This design enables targeting a single notification to multiple users — for example, an assignment announcement to all students in a section — while tracking read status individually per user. The alternative design of storing a single notification with a list of recipients embedded would make per-user read status tracking awkward and would not support efficient "unread count" queries.

3.4 Access Control Method

The access control implementation follows a middleware-first approach where authorization is fully resolved before any business logic handler receives control of a request.

Authentication Middleware: Every protected API route passes through authentication middleware that extracts and validates the JWT token from the request Authorization header. Validated tokens contain the user's ID, role, and organizational scope assignments. These values are attached to the request context for downstream use.

Role-Check Middleware: After authentication, role-check middleware evaluates whether the authenticated user's role permits the requested operation. Role checks are defined as configuration objects that enumerate which roles may perform which operation types on which resources. Requests from unauthorized roles receive a 403 response before the route handler executes.

Scope-Check Logic: Within route handlers, scope-check utility functions verify that the specific data being accessed falls within the user's organizational assignments. A teacher fetching attendance records for a section is verified to be assigned to that section. A department head fetching analytics for a department is verified to be assigned to that department. Scope violations return 403 responses with appropriate error messages.

Input Validation: All incoming request bodies are validated against Zod schemas before reaching business logic. Validation failures return structured 400 responses with field-level error details.

3.5 Workflow Methods for Key Modules

Assignment Workflow: Teachers create assignments specifying title, description, due date, scoring rubric, and audience selection — which can be section-wide or targeted to specific enrolled students. The audience selection mechanism is implemented through a flexible targeting model that supports three levels of granularity: section-wide targeting, which creates assignment records for all currently enrolled students in the section; group targeting, which creates records for a specified subset of students; and individual targeting, which creates a record for a single student. This flexibility accommodates diverse pedagogical use cases, from class-wide problem sets to personalized remediation assignments for students who require additional practice.

On creation, the system generates notification events for all targeted students through an asynchronous notification job that processes the target list and creates individual notification records. This asynchronous approach ensures that assignment creation responds immediately regardless of the size of the target audience. A section with two hundred students would require two hundred notification record insertions — processing these synchronously would introduce a multi-second delay in the assignment creation response.

Near-deadline reminder jobs are scheduled at assignment creation time using BullMQ's delayed job feature, which accepts a delay in milliseconds from the current time and dispatches the job to a worker only after that delay has elapsed. The reminder job is scheduled to execute twenty-four hours before the due date for all students who have not yet submitted. At execution time, the worker queries the submission table to identify non-submitters, generates reminder notifications for each, and dispatches them through the notification delivery pipeline.

Students access the assignment from their dashboard, review the description and rubric, and submit their response before the deadline. The submission interface supports three input modalities — file upload for document-based assignments, text editor for written responses, and code editor with language selection for programming assignments — and selects the appropriate interface based on the submission type configured by the teacher. The system records the submission timestamp and prevents further edits after the deadline unless the teacher explicitly grants a deadline extension through the extension management interface.

After the deadline, the assignment transitions to the grading phase. The teacher's grading interface presents submissions one at a time with the student's response on the left and the grading controls on the right. The grading controls include a numerical score input bounded by the maximum score, a rubric criterion selector that allows the teacher to mark which rubric elements were satisfied or violated, and a text feedback field for written comments. Rubric criterion selection is stored in structured form rather than as free text, enabling analytics on which criteria are most commonly failed across the student cohort.

Coding Submission Workflow: When a student triggers code execution in run mode, the code and selected test case input are sent directly to the Judge0 service through the backend proxy and the output is returned synchronously. The synchronous flow is appropriate for run mode because the student is actively waiting for the output and expects immediate feedback. Run mode evaluates only the selected sample test case with a shorter time limit than the full evaluation, providing quick turnaround at the cost of reduced completeness.

When a student submits for judging, the API endpoint creates a Submission record in the database with a status of PENDING and immediately enqueues a BullMQ job containing the submission ID. The endpoint responds to the client with a 202 Accepted status and the submission ID. The client uses the submission ID to join a Socket.io room and begins waiting for the verdict event. The 202 response design is an intentional departure from the conventional 200 OK response for successful POST requests, using the HTTP status code to communicate explicitly that the request has been accepted but processing is not yet complete.

The BullMQ worker picks up the job, fetches the submission record and all associated hidden test cases from the database, constructs a batch evaluation request for Judge0 containing the source code, language, and all test case inputs, and submits the batch. Judge0 processes the batch asynchronously and returns individual verdicts for each test case. The worker collects all verdicts, applies the verdict aggregation rule — the submission is Accepted only if all test cases pass; otherwise, the first failing verdict determines the submission verdict — computes runtime and memory statistics from the individual test case results, and updates the Submission record. The Socket.io event emitted after the database update contains the complete verdict information including per-test-case results, enabling the frontend to display a detailed breakdown of which test cases passed and failed.

Contest Workflow: The contest workflow introduces additional complexity compared to the standard coding submission workflow through the requirement to maintain and update standings in real time during the contest window. When a submission in contest context is evaluated and an accepted verdict is recorded, the worker must also update the ContestStanding record for the submitting student and the affected problem. The standing update computes the new cumulative score and penalty and triggers a broadcast to all participants in the contest Socket.io room.

The standing computation algorithm accounts for the order of attempts within the scoring formula. Each incorrect submission on a problem before the accepted submission adds a fixed penalty in minutes to the time penalty component. This penalty model, adapted from the ICPC contest format widely used in competitive programming, incentivizes accuracy over speed of submission and is consistent with the conventions that students who participate in external competitive programming contests will already be familiar with.

3.6 Reliability and Security Methods

Rate Limiting: API endpoints, particularly submission endpoints and authentication endpoints, are protected by rate limiting middleware that restricts the number of requests a single IP or user account may make within a sliding time window. This prevents submission flooding and brute-force authentication attacks.

Input Sanitization: All text inputs that will be stored and subsequently rendered in the UI are sanitized to prevent cross-site scripting vectors. SQL injection is prevented by the parameterized query model enforced by Prisma ORM.

File Upload Control: File uploads for assignment submissions are processed through controlled channels that validate file type, enforce size limits, and store files in isolated user-specific paths in object storage. Direct user-controlled path manipulation is prevented.

Activity Logging: All significant user actions — authentication events, grade publications, submission events, user management operations — are recorded in the ActivityLog table with timestamps and context, creating an auditable trail for security review and operational debugging.

CHAPTER 4: SYSTEM DESIGN AND WORKFLOW

4.1 User Lifecycle Workflow

The user lifecycle in RankRoom begins with account provisioning by an administrator and progresses through authentication, profile completion, role-scoped onboarding, and ongoing platform engagement. The lifecycle is designed to minimize friction at each stage while ensuring that access controls are established correctly before any data interaction occurs.

Provisioning: New user accounts are created by administrators through the user management interface, which presents a multi-step form for entering user details. The administrator specifies the user's full name, institutional email address, assigned role from the role enumeration, and organizational scope assignments appropriate for the selected role. For student accounts, scope assignment consists of selecting the section from a searchable dropdown populated with existing sections in the department. For teacher accounts, scope assignment involves selecting the subjects the teacher will be assigned to, organized by section. For department heads and class coordinators, scope assignment selects the department or section they will oversee.

Batch Provisioning: For institution-wide deployments where hundreds of student accounts must be created at the beginning of a semester, the system supports batch provisioning through CSV upload. The administrator uploads a CSV file containing columns for name, email, role, and scope identifiers. The system validates each row for completeness and correctness before processing, rejecting the upload if any row contains an invalid scope reference. Validated rows are processed in a background job that creates accounts sequentially and dispatches onboarding notifications in batches to avoid overwhelming the email delivery service.

Authentication: Users authenticate through a JWT-based flow. The login form accepts the institutional email address and the current password. The server retrieves the User record by email, verifies the submitted password against the stored hash using a constant-time comparison function that prevents timing attacks, and on successful verification issues a

signed access token and a refresh token. The access token contains the user's ID, role, and scope assignments encoded as JWT claims and is signed using HMAC-SHA256 with the server's secret key. The access token has a fifteen-minute expiration. The refresh token is a random opaque identifier stored in the database with an expiration of thirty days, enabling session continuation without requiring re-authentication.

Profile Synchronization: On first login after password change, users are directed to the profile completion flow. The flow collects biography, contact information, and for students, GitHub and LeetCode usernames. These external platform usernames are optional but unlock the combined activity heatmap feature on the student profile page. When a GitHub username is provided, the system makes an asynchronous request to the GitHub public API to verify that the username exists and fetches the contribution calendar data for the current year. When a LeetCode username is provided, the system fetches the public statistics — total problems solved by difficulty, submission count, and contest rating — from the LeetCode public API. Both data sets are cached in the StudentProfile entity and refreshed periodically by a scheduled background job, ensuring that the profile page displays reasonably current data without making API calls on every page load.

Session Management: The frontend stores the access token in memory and the refresh token in an HttpOnly cookie. Before each API request, the frontend checks whether the access token is within two minutes of expiration and, if so, makes a silent refresh request using the refresh token. The refresh endpoint validates the refresh token against the database, verifies that the associated user account is still active, and issues a new access token with fresh scope assignments from the database. This refresh mechanism ensures that scope changes — such as a teacher being reassigned to a different section — take effect within one token lifetime without requiring explicit re-login.

Account Deactivation: When a user leaves the institution or is suspended, administrators can deactivate the account through the user management interface. Deactivation sets the is_active flag on the User record to false. The authentication middleware checks this flag after token validation and rejects requests from deactivated accounts with a 401 Unauthorized response. Existing active sessions are invalidated by revoking the refresh token in the database, ensuring that the deactivated user cannot use an existing session to continue accessing the platform after deactivation.

4.2 Academic Workflow

Department and Section Setup: The institutional administrator creates department records and within each department creates sections representing academic cohorts. Each section has a name, a year, and an associated class coordinator.

Subject Mapping: Subjects are created and mapped to sections with a defined number of sessions per week and a minimum attendance threshold. Teachers are assigned to subject-section combinations, defining their grading and attendance authority scope.

Enrollment: Students are enrolled in sections, which triggers the creation of their section membership, links them to all subjects within that section, and initializes their attendance and grade tracking records.

Attendance Recording: Teachers initiate attendance sessions from their dashboard for a specific subject and section. The session is timestamped and students are marked present or absent. Absent students may be marked present later if they submit a valid excuse within a configurable window, subject to teacher approval.

Grade Entry: Teachers enter component-wise grades — quiz marks, assignment marks, midterm marks, end-term marks — for each enrolled student. The system applies the configured grading formula to compute composite scores and grade letter equivalents. Students can view their grades as soon as they are published by the teacher.

4.3 Assignment Workflow

Assignment management in RankRoom follows a structured lifecycle designed to reduce teacher overhead while providing students with clear expectations, timely feedback, and transparent grading.

Creation Phase: Teachers access the assignment creation interface from their subject dashboard. They define the assignment title, detailed description with optional rich text formatting, the submission type — file upload, text response, or code submission — the maximum score, the grading rubric, the due date and time, and the penalty policy for late submissions. The audience selection step allows teachers to target the entire section or specific enrolled students.

Notification Phase: On assignment publication, the system creates notification records for all targeted students with the assignment title, due date, and a direct link to the submission interface. Near-deadline reminder jobs are scheduled to trigger 24 hours before the due date for students who have not yet submitted.

Submission Phase: Students access the assignment from their dashboard, review the description and rubric, and submit their response before the deadline. The system records the submission timestamp and prevents further edits after the deadline unless the teacher explicitly grants an extension.

Grading Phase: After the deadline, the assignment transitions to the grading phase. Teachers see a list of all submissions organized by student, with submission timestamps and completion status. They review each submission, enter a numerical score, select applicable rubric criteria, and write written feedback. Submissions can be graded individually or the teacher can navigate through submissions sequentially.

Publication Phase: Teachers publish grades either individually or in batch. On publication, each student receives a notification with their score and feedback visibility. Published grades feed into the grade analytics pipeline and update the student's performance summary.

4.4 Coding Problem Workflow

Problem Authoring: Teachers and administrators create coding problems through a structured authoring interface. They define the problem title, difficulty level, topic tags, problem statement with input-output format specifications, constraints on input size and value ranges, sample test cases visible to students, and hidden test cases used for judging. Optional components include structured hints that students can reveal progressively and an editorial explaining the optimal solution approach.

Scope Configuration: Problems can be scoped globally — visible to all students on the platform — or scoped to specific sections, making them visible only to enrolled students of those sections. This allows teachers to create curriculum-specific practice problems alongside platform-wide competitive problems.

Run Mode: Students work on problems in a split-pane interface with the problem description on the left and a Monaco code editor on the right. In run mode, the student selects a test case, writes or modifies code, and clicks Run. The frontend sends the code directly to the Judge0 service through the backend proxy and displays the output alongside the expected output for comparison. Run mode evaluations are not recorded as submissions.

Submit Mode: When the student is satisfied with their solution, they click Submit. This creates a Submission record in the database with status Pending, enqueues a BullMQ job containing the submission ID, and immediately returns a submission confirmation to the client. The client subscribes to a Socket.io room for this submission.

Evaluation Pipeline: A BullMQ worker picks up the submission job, fetches the submission and associated problem test cases from the database, constructs a batch evaluation request for Judge0 containing all hidden test cases, submits the batch, polls for completion, receives all test case verdicts, computes the final submission verdict — Accepted only if all test cases pass, otherwise the first failing verdict — records runtime and memory statistics, updates the Submission record status, and emits a Socket.io event to the client room delivering the complete verdict.

4.5 Contest Workflow

Contest Creation: Organizers create contests by defining the title, description, start time, end time, registration deadline, problem set selection, and scoring configuration — including whether the contest uses a standard penalty model or a custom scoring formula.

Registration Phase: Students register for the contest within the registration window. On registration, a ContestRegistration record is created and the student is added to the contest's Socket.io room for live standing updates.

Contest Execution: During the contest window, students access the contest interface which shows all problems, their submission history for each problem, and a real-time standings display. Submissions are evaluated through the same pipeline as regular coding submissions, but verdict delivery also triggers a ContestStanding update and a broadcast to all contest participants.

Standing Computation: Contest standings are computed using total score as the primary rank determinant and cumulative time penalty as the tiebreaker. The time penalty for a correct submission is the submission time elapsed since contest start plus a fixed penalty for each incorrect submission on the same problem before the accepted submission.

Result Publication: At contest end, the organizer reviews the final standings and publishes results. Publication freezes the standings, records the final positions in each participant's profile, awards platform points based on rank, and dispatches result notifications to all participants.

4.6 Hackathon Workflow

Hackathon Setup: The workflow starts when an organizer (admin, department head, teacher, or faculty) creates a hackathon event on the platform. During setup, they define the title, description, dates, registration deadline, schedule, prizes, and participation mode (online, offline, or hybrid). Eligibility criteria such as department, section, year, academic performance, coding activity, required skills, or prior participation can also be configured. Team size limits are defined to ensure structured and transparent event management.

Registration and Team Formation: After publication, eligible students can view and register for the hackathon. The system validates eligibility before allowing participation. Students may register individually or as teams. In team-based events, one student acts as the team captain, creates the team, and invites others to join. Invited members must accept before becoming confirmed participants. Duplicate registrations and multiple team participation in the same event are restricted to maintain fairness.

Participation Phase: Once registration closes, the event moves into the participation phase. For offline events, organizers can manage attendance, check-ins, and team records. For online or hybrid events, participants submit projects through GitHub repositories, demo links, presentations, or documentation. Submissions can be updated until the deadline, while timestamps are recorded for accountability. This centralizes event management within one platform.

Judging and Results: After the event ends, judges evaluate submissions based on criteria like innovation, technical complexity, relevance, and presentation. Scores and remarks are recorded in the system, which calculates final rankings and identifies winners. Achievements can be added to student profiles, and certificates may be generated automatically, reducing administrative effort.

4.7 Notification and Reminder Workflow

Event-Triggered Notifications: The notification system keeps users updated about important platform activities. Events such as assignment publication, grades, contest announcements, hackathon confirmations, team invitations, submissions, verdicts, and results automatically trigger notifications. Each notification follows predefined templates and can target individuals, sections, departments, role groups, or the entire platform. This ensures relevant communication for each user.

Notification Delivery and Tracking: Notifications are stored in the database with recipient details and read/unread status, allowing users to maintain a personal notification history. Students can track assignments, contests, hackathons, verdicts, and grades, while faculty and administrators can communicate updates efficiently. Real-time delivery through Socket.IO ensures instant updates for active users.

Scheduled Reminders: The platform also supports reminders for deadlines and events. Assignment reminders can be sent before due dates, contest reminders before start times, and hackathon reminders before registration deadlines or submission cutoffs. These reminders help improve participation and reduce missed deadlines.

Reminder Processing and Reliability: Reminder jobs are stored in the background job system to ensure reliability even after server restarts. The worker checks pending reminders, identifies due tasks, resolves recipients, and sends notifications. Deduplication logic prevents repeated reminders, making the workflow reliable for institutional use.

4.8 Analytics Workflow

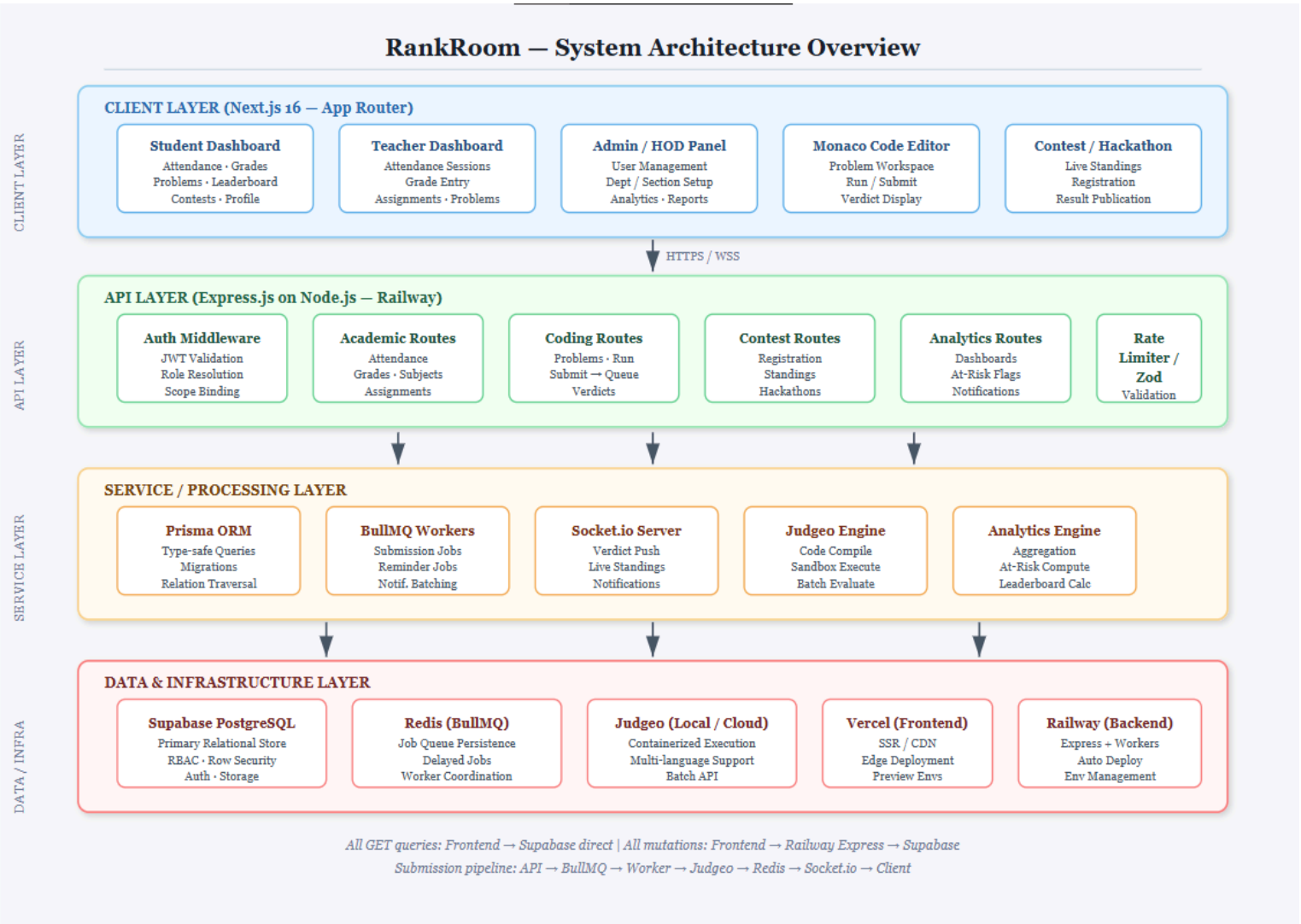
Data Aggregation: The analytics pipeline aggregates data from all module databases. Attendance aggregation computes present count and total session count per student per subject, deriving the attendance percentage. Grade aggregation applies the subject grading formula to component marks to produce composite scores and letter grades. Assignment aggregation counts submitted, graded, and unsubmitted assignments per student. Coding aggregation counts accepted submissions, total problems attempted, and categories of problems solved.

At-Risk Detection: At-risk identification rules are configurable per institution. Default rules flag students with attendance below the minimum threshold in any subject, students with two or more unsubmitted assignments in the past two weeks, and students with composite scores below the minimum passing threshold in any subject. Students matching any at-risk rule receive an at-risk flag in the analytics dashboard visible to their teachers and coordinators.

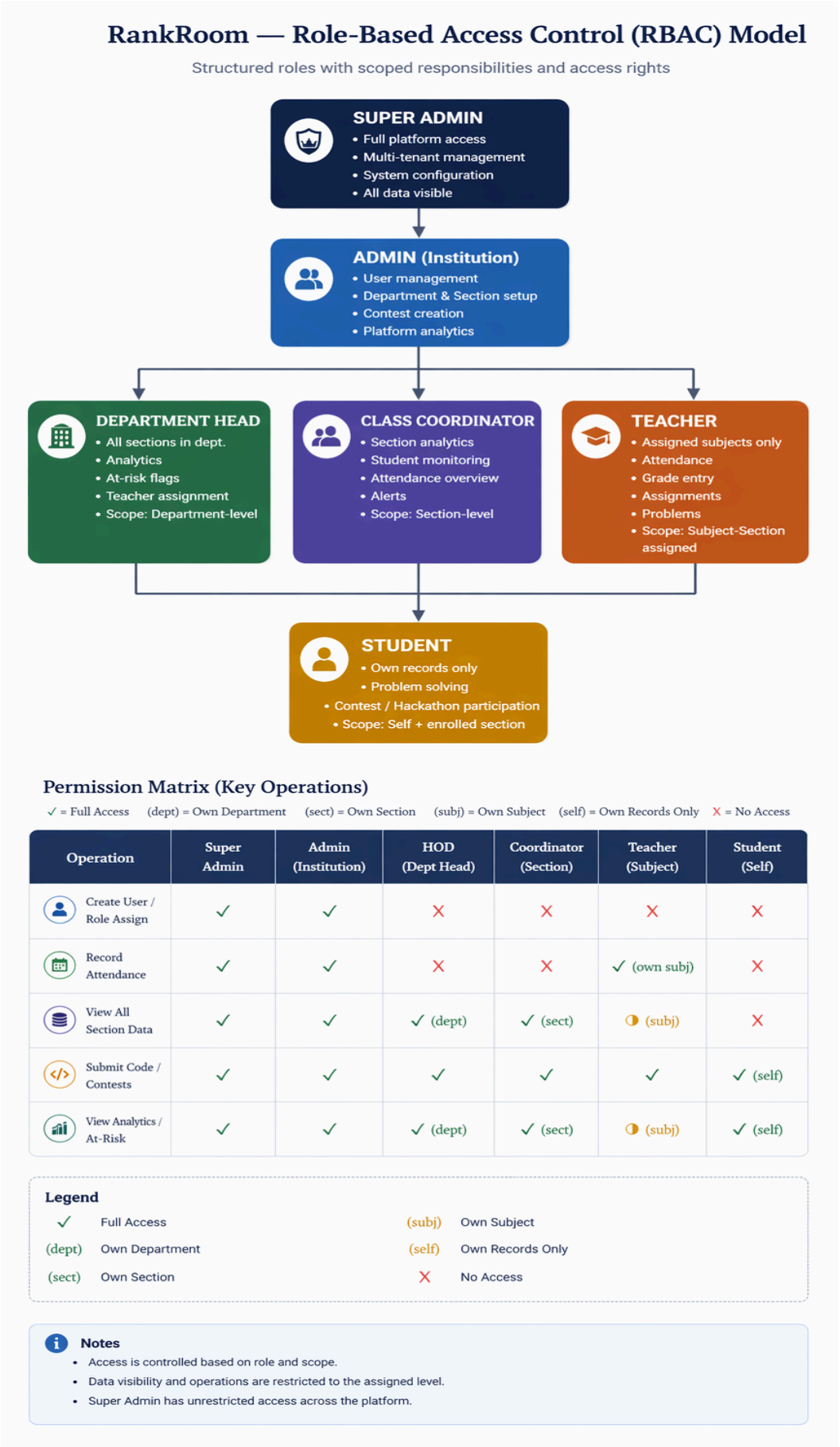
Dashboard Delivery: Analytics dashboards are served through role-scoped API endpoints. Student dashboards present personal metrics with trend visualization. Teacher and coordinator dashboards present section-level aggregate views with drill-down capability to individual student data. Department head dashboards present department-level summaries. Admin dashboards present platform-wide enrollment, activity, and performance summaries.

4.9 Architectural Diagrams

4.9.1 System Architecture

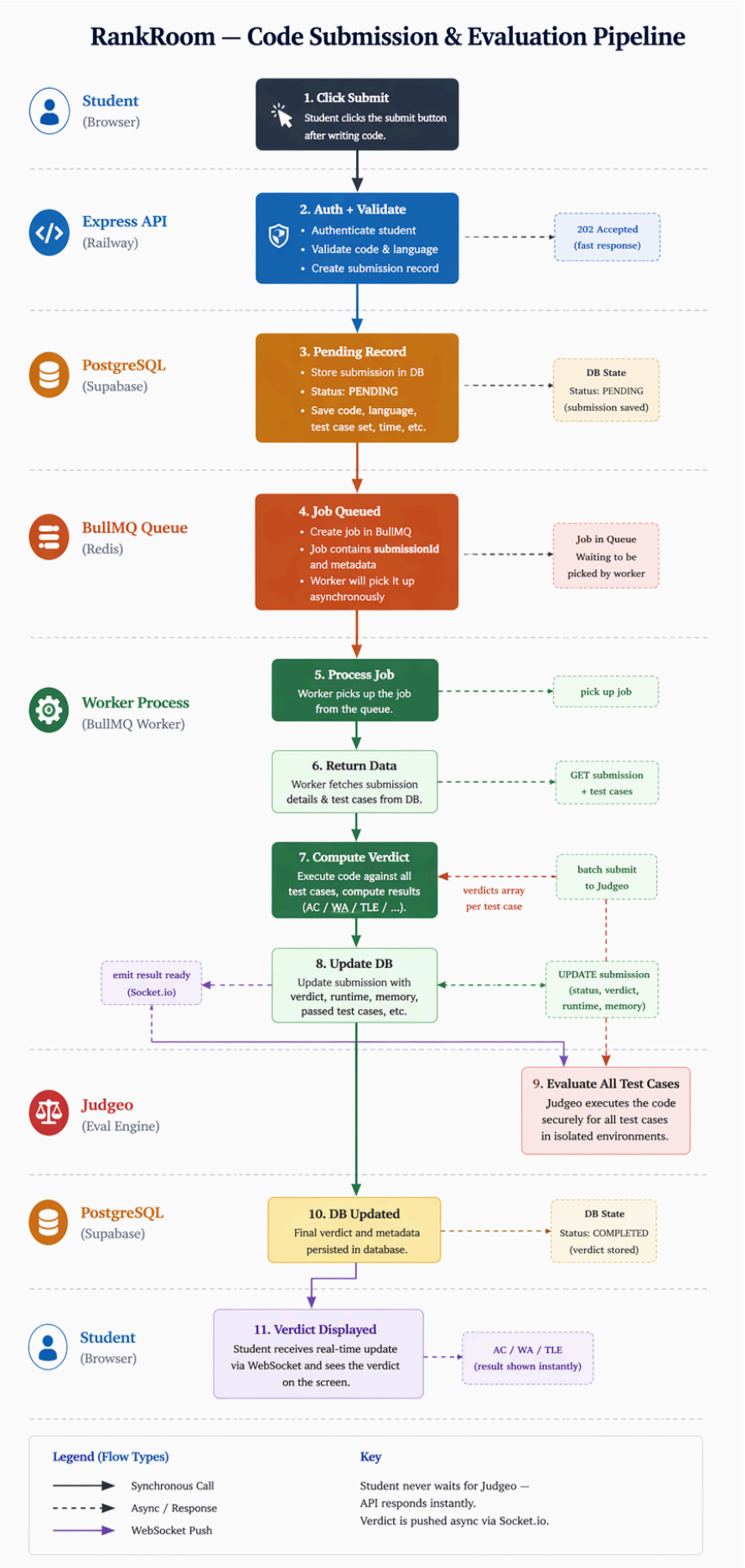


4.9.2 RBAC Model

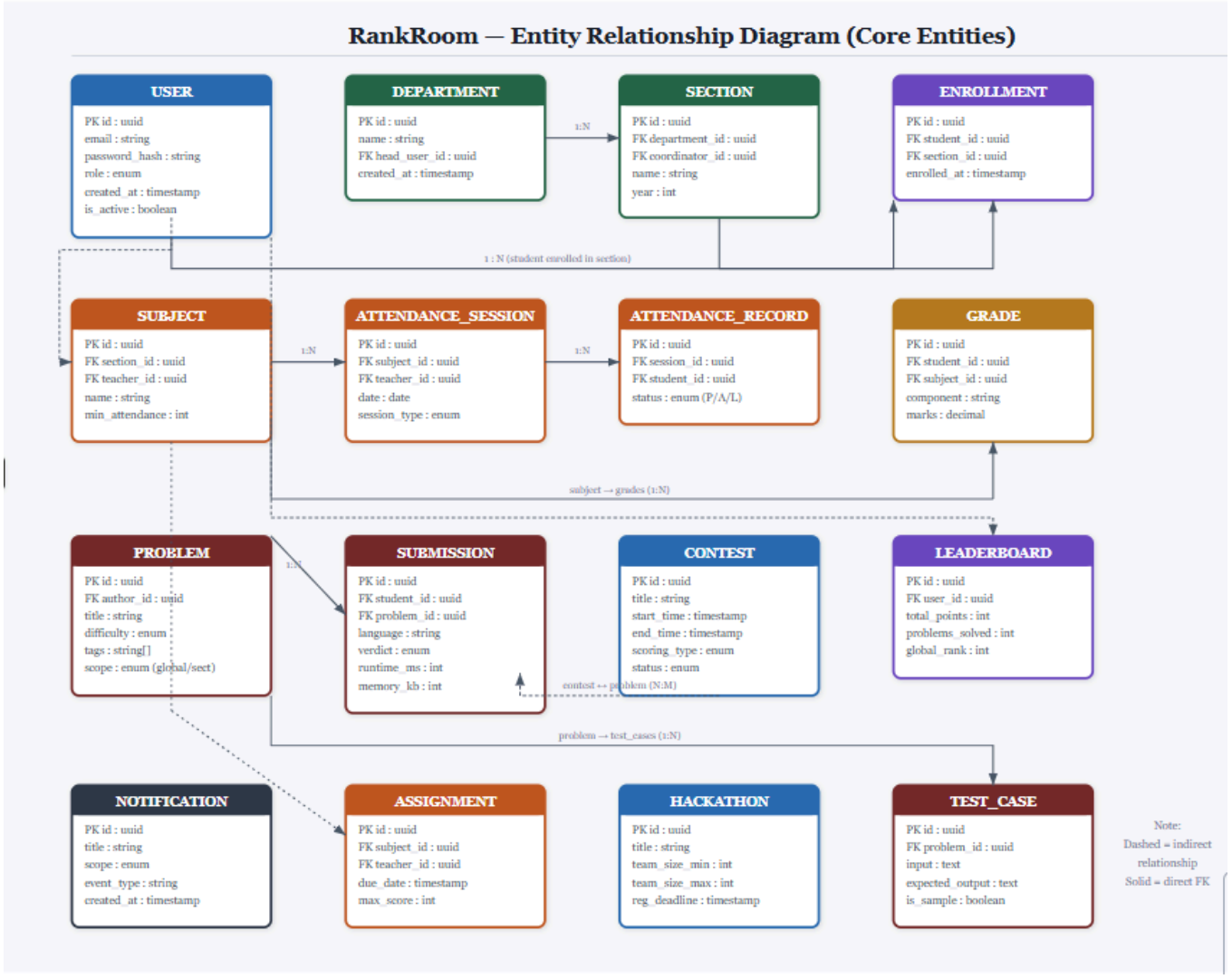


Operation	Super Admin	Admin (Institution)	HOD (Dept Head)	Coordinator (Section)	Teacher (Subject)	Student (Self)
Create User / Role Assign	✓	✓	✗	✗	✗	✗
Record Attendance	✓	✓	✗	✗	✓ (own subj)	✗
View All Section Data	✓	✓	✓ (dept)	✓ (sect)	◑ (subj)	✗
Submit Code / Contests	✓	✓	✓	✓	✓	✓ (self)
View Analytics / At-Risk	✓	✓	✓ (dept)	✓ (sect)	◑ (subj)	✓ (self)

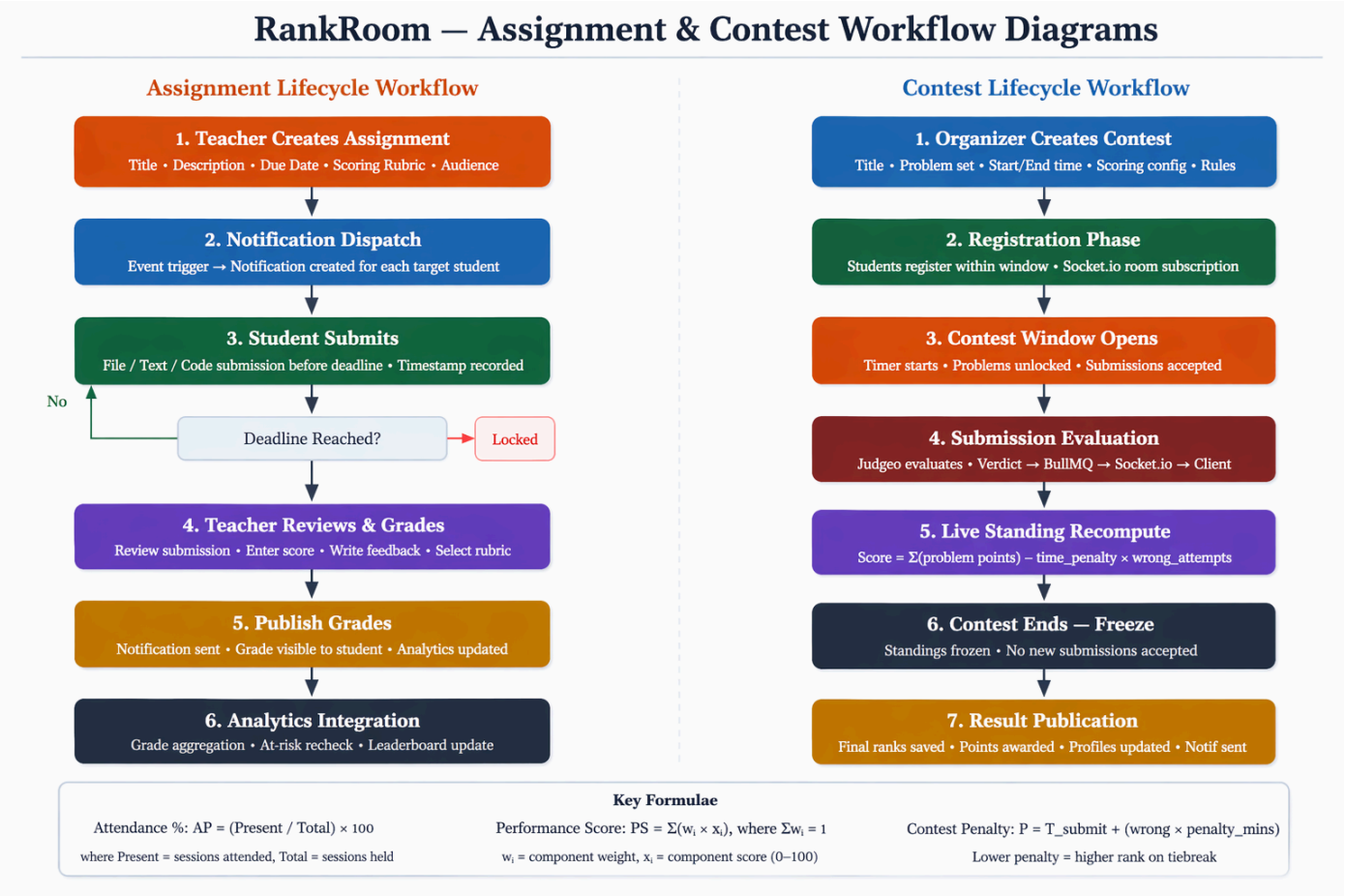
4.9.3 Code Submission & Evaluation Pipeline



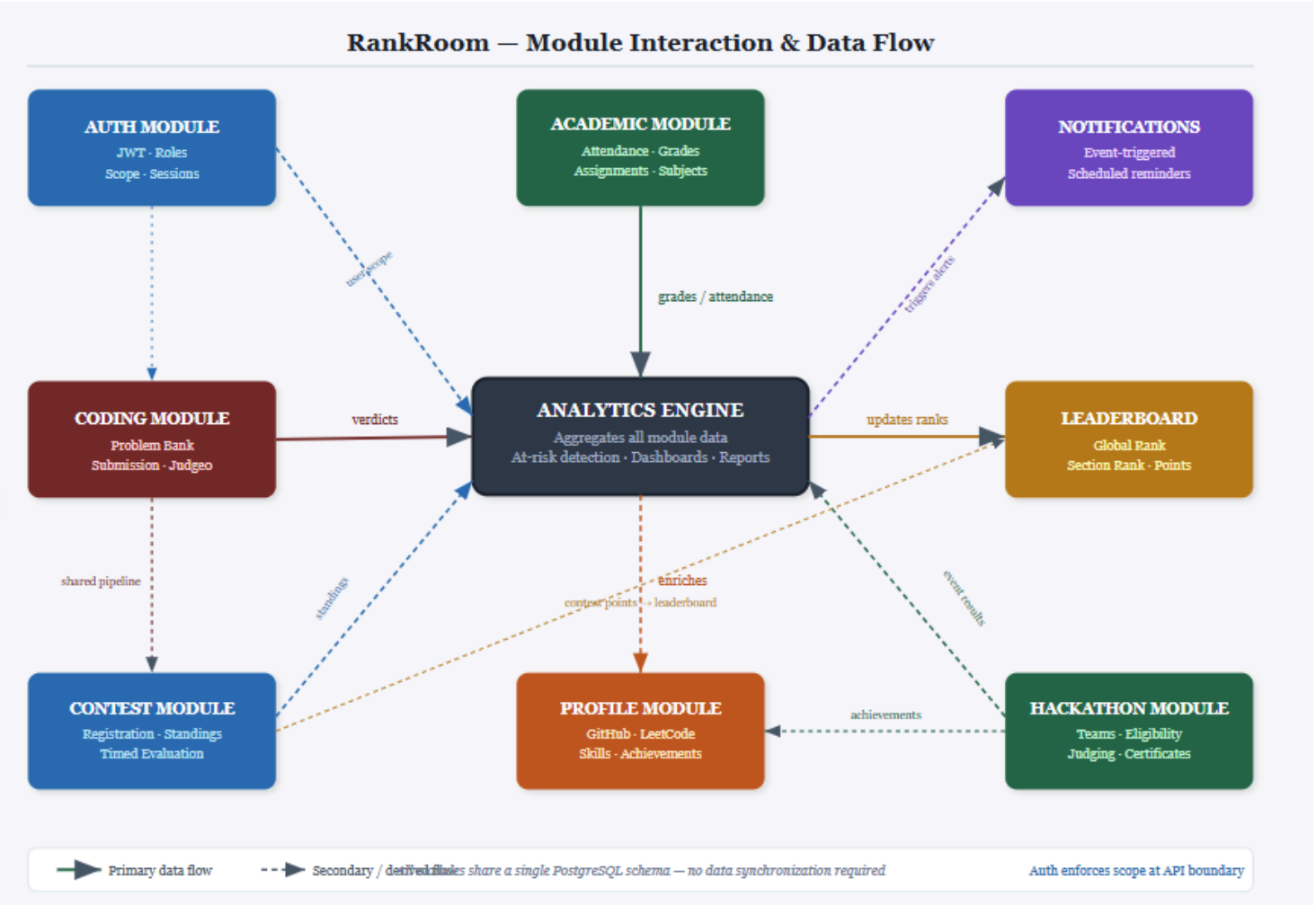
4.9.4 E.R. Diagram



4.9.5 Assignment and Contest Workflow



4.9.6 Module Interaction and Data Flow



CHAPTER 5: ANALYSIS AND DISCUSSION

5.1 Functional Analysis

The integrated design of RankRoom delivers several functional improvements over the fragmented multi-system approach that characterizes most institutional academic environments.

The consolidation of attendance, grades, assignments, coding, contests, hackathons, and notifications within a single system boundary eliminates the manual reconciliation overhead associated with maintaining parallel systems. Faculty who previously needed to export data from an attendance portal, cross-reference it with a grade spreadsheet, and separately check assignment submission logs can now access all of this information from a single dashboard with a consistent interface and a unified data model.

Role-scoped dashboards ensure that each user encounters only the data and operations relevant to their responsibilities. A student dashboard is uncluttered by administrative controls. A teacher dashboard surfaces pending grading tasks and upcoming attendance sessions without exposing the administrative and analytics views that are irrelevant to their workflow. This role-specific curation reduces cognitive load and improves task completion efficiency.

The assignment lifecycle management — covering creation, targeting, submission, grading, and feedback — removes the need for ad-hoc communication through email or messaging applications. All assignment-related communication happens through structured, tracked platform interactions, creating a complete audit trail for academic integrity purposes.

The coding problem workflow, integrating the Monaco Editor, Judge0 evaluation engine, BullMQ queue, and Socket.io verdict delivery, provides a professional-grade competitive programming experience within the institutional context. Students experience the same quality of immediate feedback that external platforms provide, without needing to leave the institution's system to develop coding skills.

5.2 Performance and Scalability Discussion

The asynchronous submission pipeline is the most critical performance design decision in the system. By decoupling submission acceptance from submission evaluation, the API endpoint that receives a student's code submission can respond within milliseconds — creating the submission record and enqueueing the job — even when the evaluation pipeline is processing a large backlog of submissions during a contest.

BullMQ's Redis-backed queue provides durability guarantees that prevent submission loss even in the event of worker process failures. If a worker crashes during job processing, BullMQ's job locking mechanism ensures the job is picked up by another worker after the lock timeout expires. This fault tolerance is essential during contest events when submission loss would be unacceptable.

Socket.io verdict delivery eliminates the need for client-side polling, which would create significant unnecessary load on the API server during contests when hundreds of students might be simultaneously waiting for verdicts. The room-based subscription model ensures that verdict events are delivered only to the relevant client, preventing unnecessary network traffic.

The Supabase-hosted PostgreSQL database provides horizontal read scalability through read replicas if needed, while the Prisma ORM's query optimization and connection pooling manage connection overhead efficiently for the expected concurrent user load.

5.3 Engagement Discussion

The leaderboard and contest participation mechanisms serve as the primary behavioral engagement drivers in RankRoom. Leaderboard visibility — both global rankings and section-level rankings — creates social comparison pressure that research consistently identifies as a powerful driver of voluntary platform engagement.

The skill radar visualization on student profile dashboards provides competence feedback across problem-solving categories, giving students a visual representation of their strengths and weaknesses. This connects to Self-Determination Theory's competence dimension, which predicts that competence feedback that is granular and actionable drives stronger intrinsic motivation than aggregate score-only feedback.

The combined activity heatmap that aggregates GitHub contributions, LeetCode submissions, and RankRoom coding activity creates a unified view of a student's technical engagement across platforms. This integration is a meaningful innovation over both institutional LMS systems and external coding platforms, providing a holistic engagement narrative that neither offers alone.

The growth coach feature — which analyzes a student's performance profile and generates personalized practice recommendations — extends engagement beyond reactive tracking into proactive guidance, addressing the research finding that personalized feedback produces stronger learning outcomes than generic content delivery.

5.4 Academic Monitoring Discussion

Attendance and grade analytics within RankRoom enable a level of real-time academic monitoring that is practically impossible with disconnected systems. The at-risk detection system operationalizes the research on early warning indicators by continuously evaluating student data against configurable threshold rules and surfacing flags in faculty dashboards.

The correlation between academic indicators and coding engagement metrics is a uniquely valuable capability enabled by the integrated platform model. A teacher can observe that a student's attendance decline coincides with reduced coding activity and declining assignment submission rates — a pattern that may indicate a personal difficulty requiring support intervention — without needing to manually query multiple systems and align the data temporally.

Section-level analytics enable class coordinators to compare cohort performance across subjects, identify subjects where the section collectively struggles, and evaluate whether specific teaching or grading patterns are producing unexpected outcomes. This cross-subject visibility is unavailable in subject-siloed grade recording systems.

5.5 Security and Governance Discussion

The multi-dimensional RBAC model — combining role type checks and organizational scope checks at every API boundary — provides a strong security baseline appropriate for an institution handling sensitive student academic data.

The scope enforcement prevents cross-section data leakage, which is a significant risk in educational systems where teachers may inadvertently or intentionally access data beyond their authorized scope. By enforcing scope at the API layer rather than relying on frontend navigation controls, the system ensures that direct API access — whether from an API client, a browser developer tool, or a script — cannot circumvent the scope boundaries.

Activity logging across significant user actions creates an audit trail that supports institutional governance requirements. Any grade modification, user role change, or attendance correction is logged with the acting user's identity and timestamp, enabling post-hoc review in cases of dispute.

JWT-based authentication with short-lived access tokens and refresh token rotation limits the exposure window of compromised credentials while maintaining a smooth user experience through transparent token renewal.

5.6 Limitations

The quality of analytics outputs depends on the consistency and timeliness of faculty data entry. If teachers delay recording attendance or publishing grades, the at-risk detection system operates on incomplete data and may fail to flag students who are genuinely at risk. Institutional adoption of RankRoom as the authoritative system for these workflows, rather than a parallel system to existing tools, is a prerequisite for the analytics value proposition.

The real-time modules — verdict delivery, live contest standings — depend on stable network connectivity. In environments with intermittent connectivity, the Socket.io connection may drop, causing users to miss real-time updates. The frontend implements reconnection logic and provides polling fallbacks, but the user experience degrades under poor network conditions.

The current implementation targets single-institution deployment. Multi-institution tenancy — enabling inter-college contests or shared problem banks — requires additional architectural work around data isolation and cross-institution identity management.

Large-scale load testing under realistic contest-time submission volumes has been conducted at moderate scale. Further testing at larger student population sizes would be needed before deployment at institutions with thousands of concurrent users.

5.7 Overall Discussion

RankRoom demonstrates that the technical integration of academic administration and competitive skill engagement is achievable within a single platform using established open-source technologies. The system's layered architecture, role-scoped data model, and asynchronous processing pipeline address the primary technical challenges of building a unified institutional platform without requiring proprietary infrastructure.

The functional integration — bringing attendance, grades, assignments, coding problems, contests, hackathons, and analytics under one authentication boundary and one data model — delivers meaningful operational improvements over the disconnected systems it replaces. The engagement mechanisms, grounded in gamification research, provide a motivation framework that sustains student interaction with both mandatory academic processes and optional skill-building activities.

CHAPTER 6: CONCLUSION AND FUTURE WORK

6.1 Conclusion

This project presented the complete design and implementation of RankRoom, a unified academic and competitive learning platform developed for higher educational institutions. The system successfully integrates attendance management, grade recording and publication, assignment lifecycle management, a competitive coding problem platform with automated judging, contest and hackathon event management, leaderboard-driven engagement mechanics, real-time notification delivery, and comprehensive analytics dashboards within a single role-secured, scope-enforced application.

The fundamental problem motivating this work — the fragmentation of academic and skill-development workflows across disconnected systems — is directly addressed by the platform's unified data model and integrated workflow design. Faculty no longer need to maintain attendance in one system, grades in another, and separately monitor student coding engagement on external platforms. Students no longer experience a disjointed academic journey across multiple tools with no unified identity or progression narrative.

The technical architecture — combining a Next.js frontend, Express.js API layer, PostgreSQL via Supabase, Prisma ORM, BullMQ asynchronous job processing, Redis persistence, Socket.io real-time communication, and Judge0 code evaluation — demonstrates that a platform of this complexity and capability is achievable with open-source components and modern web engineering practices.

The RBAC model, with its combination of role-type and organizational-scope enforcement at the API boundary, provides the security and governance foundation necessary for institutional data handling. The analytics pipeline, correlating academic and coding engagement indicators, provides actionable faculty insights that are unavailable from any combination of the disconnected systems RankRoom replaces.

RankRoom provides a practical and replicable model for educational institutions seeking to bridge academic administration and employability-focused skill development within a coherent, maintainable, and extensible platform.

6.2 Future Work

Adaptive Recommendation Engine: A machine learning module that analyzes individual student problem-solving patterns, topic performance on assessments, and historical engagement data to recommend specific practice problems, study materials, and contest participation strategies personalized to each student's current skill profile and learning objectives.

Predictive Risk Scoring: An expansion of the at-risk detection system to incorporate predictive modeling using historical semester data. Rather than threshold-based rule evaluation, a trained model would predict the probability of academic difficulty based on early-semester behavioral signals, enabling earlier and more targeted intervention.

Plagiarism and Code Similarity Detection: Integration of code similarity analysis for both assignment submissions and coding problem solutions at scale. This requires computing pairwise similarity across large submission sets efficiently and presenting suspected similarity cases to faculty for review, with configurable sensitivity thresholds.

Multi-Institution Tenancy: Architectural extension to support multiple institutions sharing the same platform infrastructure while maintaining complete data isolation between institutions. This would enable inter-college contests, shared problem bank access, and cross-institution leaderboards while preserving institutional data governance boundaries.

Native Mobile Applications: iOS and Android applications providing low-friction access to notifications, attendance viewing, assignment deadline tracking, and leaderboard checking. Mobile applications with push notification support would significantly improve engagement with time-sensitive features.

Extended Reporting and Export: Support for generating accreditation-format reports, Excel exports of attendance and grade data, and PDF certificates for contest and hackathon participation. These exports would reduce the manual effort required for institutional reporting and accreditation processes.

Offline-First Support: Progressive Web App capabilities enabling core read operations — viewing assignment details, reading problem statements, reviewing past submissions — to function in low or no connectivity environments, with synchronization on connection restoration.

AI-Powered Feedback: Integration of AI-generated feedback for assignment submissions and coding problem solutions, providing students with immediate preliminary feedback while teachers complete manual grading, and offering solution quality commentary beyond the binary pass/fail verdict of automated test case evaluation.

APPENDIX A

LIST OF ABBREVIATIONS

API: Application Programming Interface

RBAC: Role-Based Access Control

LMS: Learning Management System

DBMS: Database Management System

ORM: Object Relational Mapping

UI: User Interface

UX: User Experience

KPI: Key Performance Indicator

CRUD: Create, Read, Update, Delete

JWT: JSON Web Token

CI/CD: Continuous Integration and Continuous Deployment

PII: Personally Identifiable Information

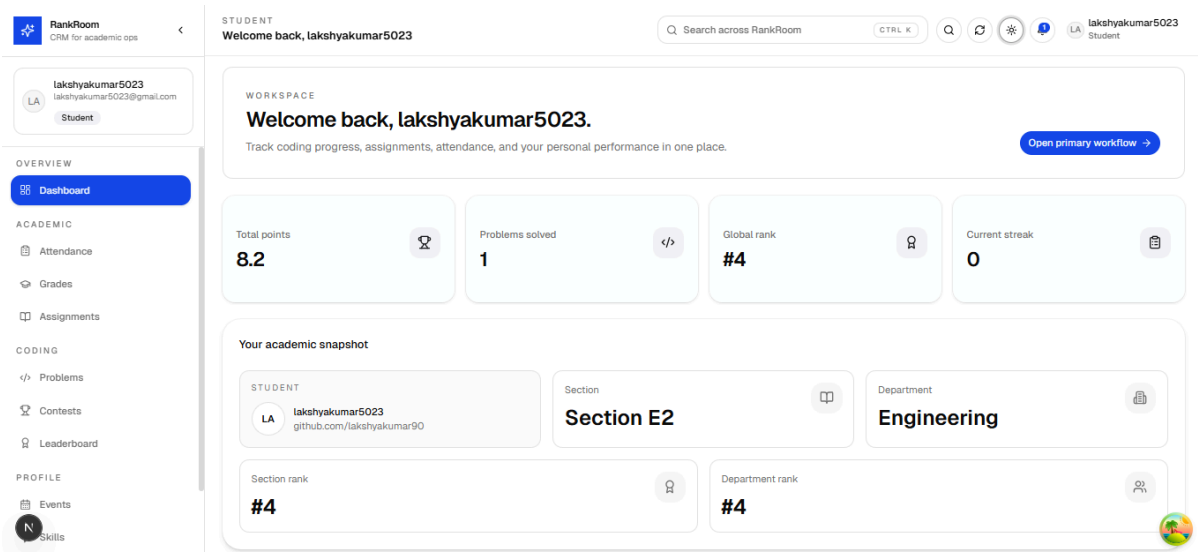
AI: Artificial Intelligence

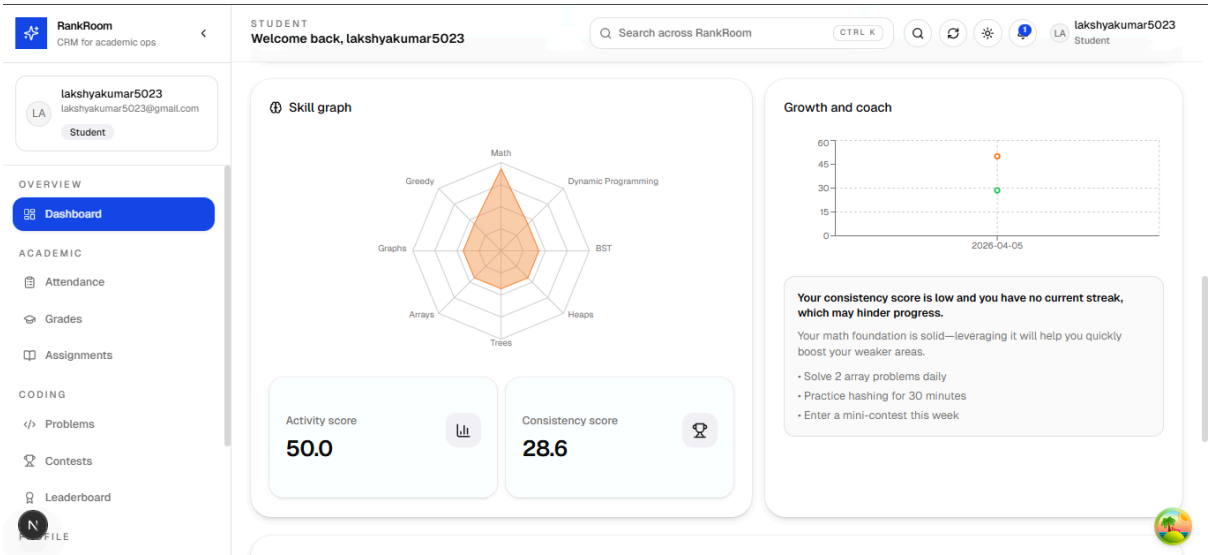
CGPA: Cumulative Grade Point Average

XP: Experience Points

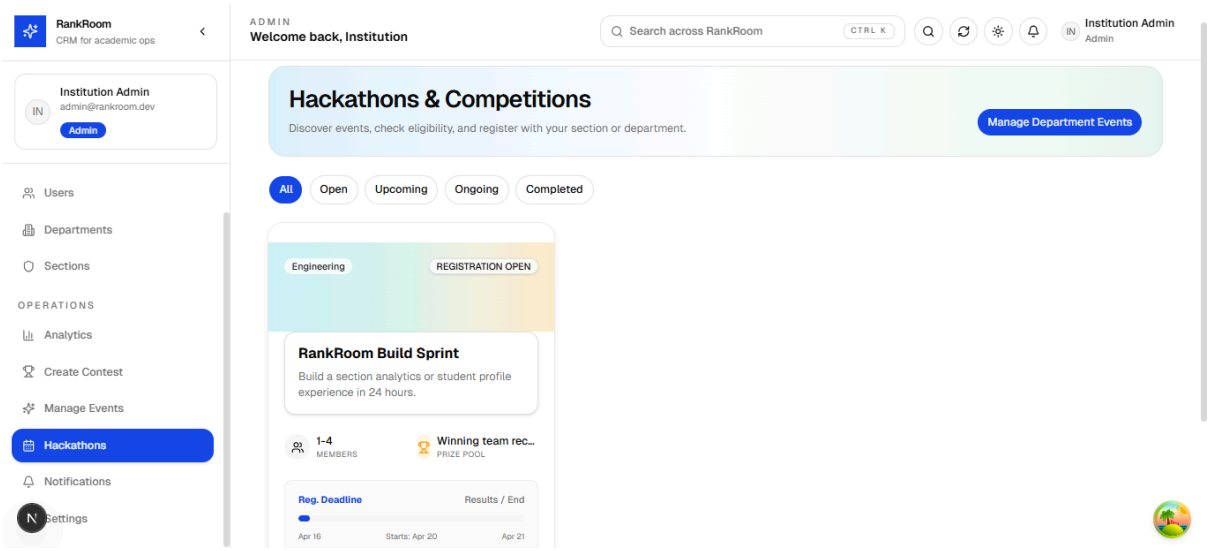
LIST OF FIGURES

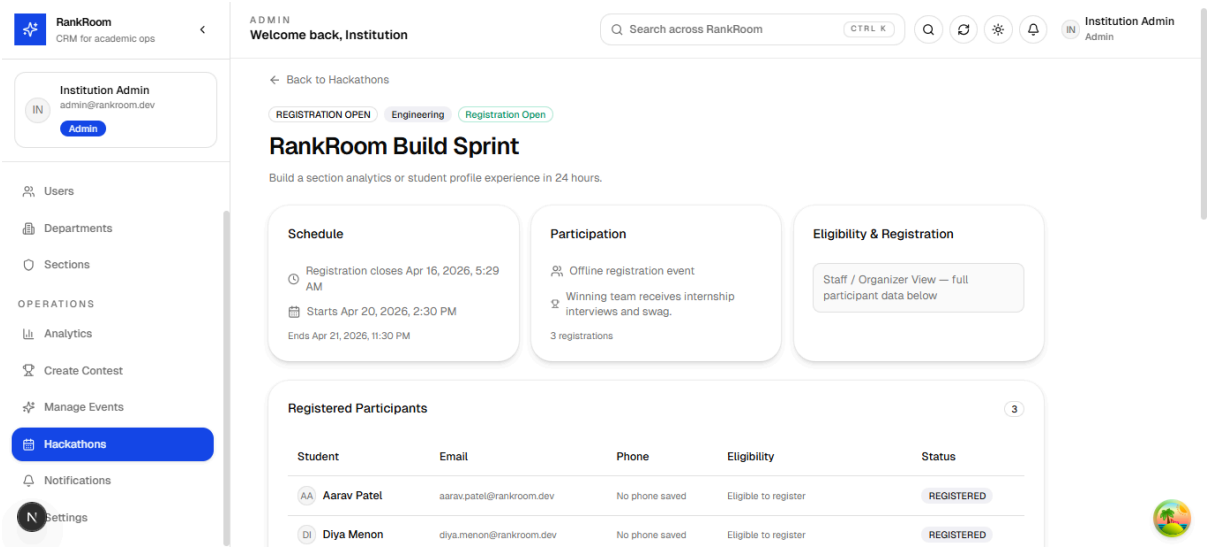
Dashboard Interface



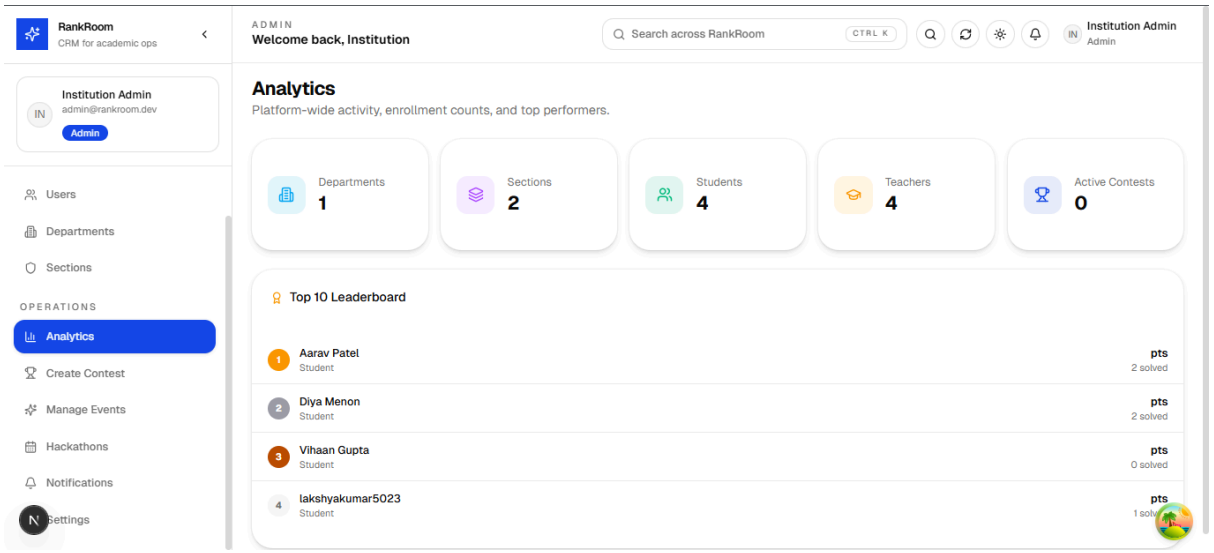


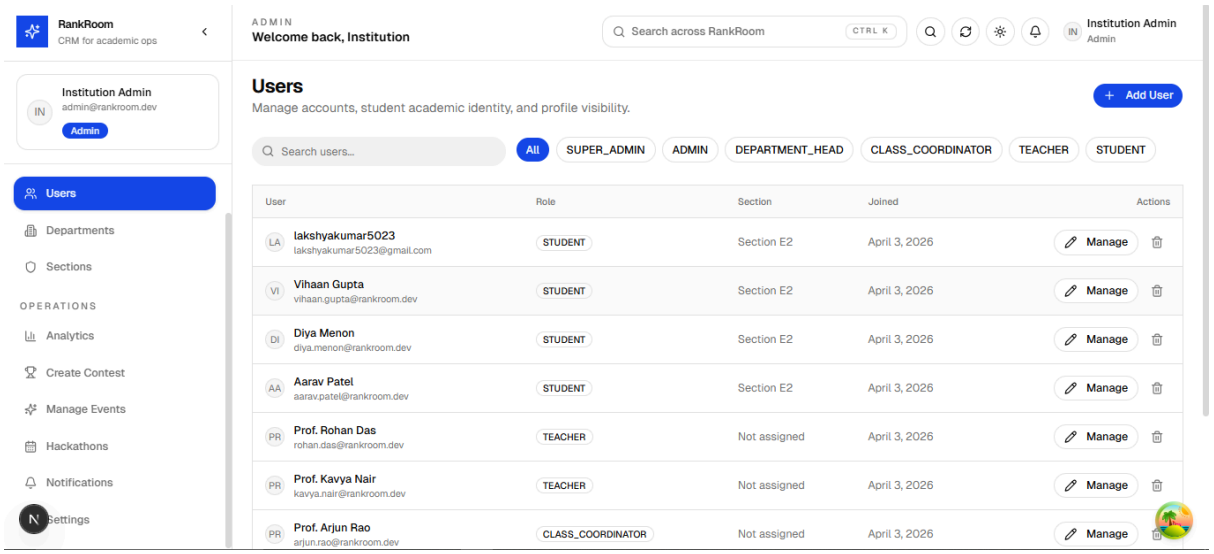
Hackathons and Events



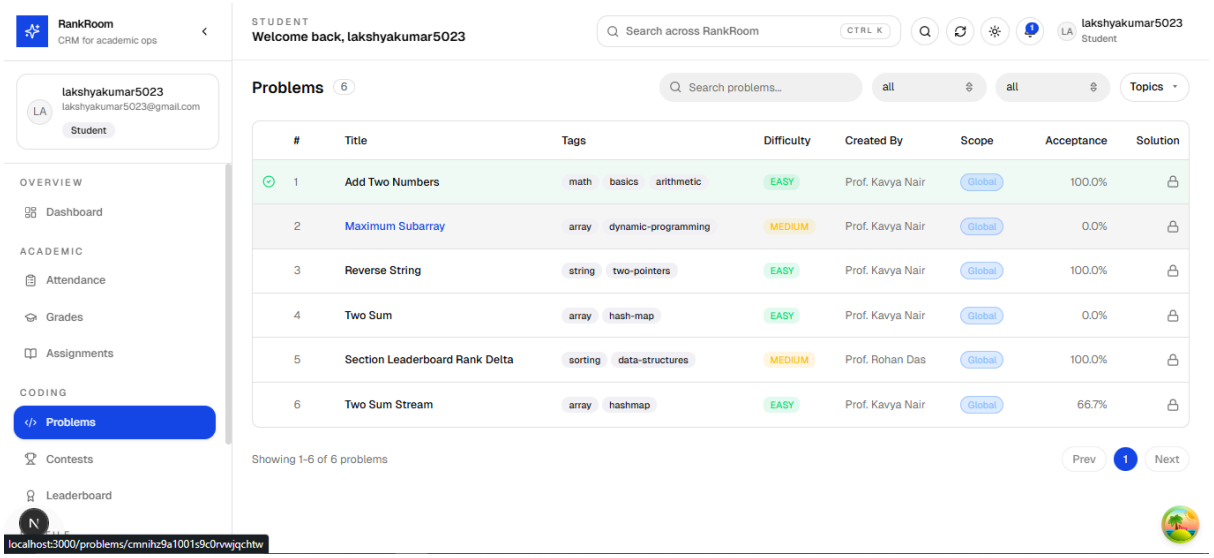


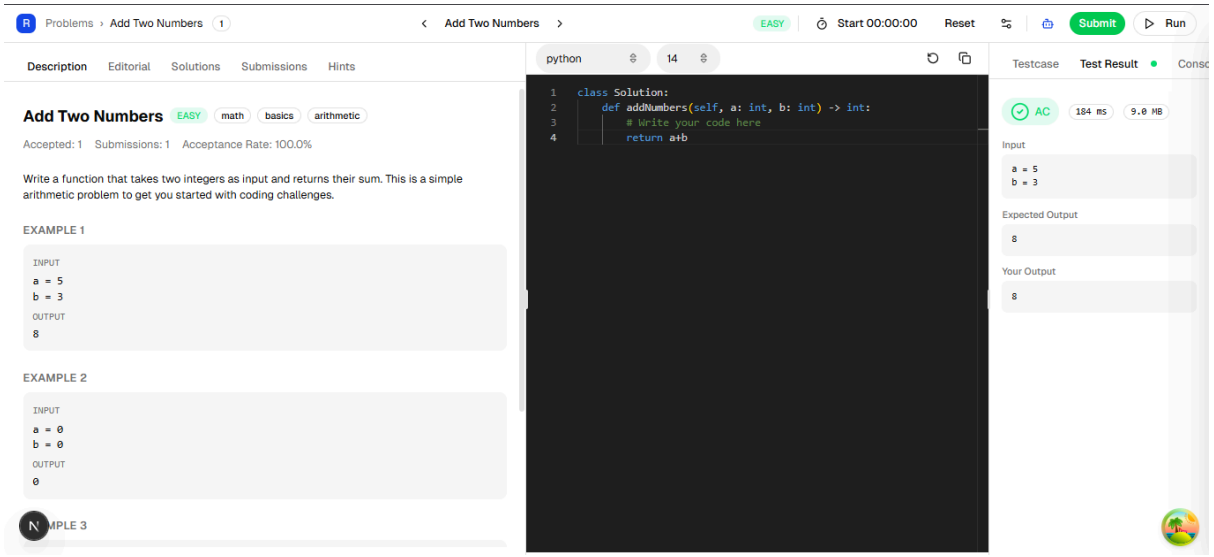
Analytics & Users overview



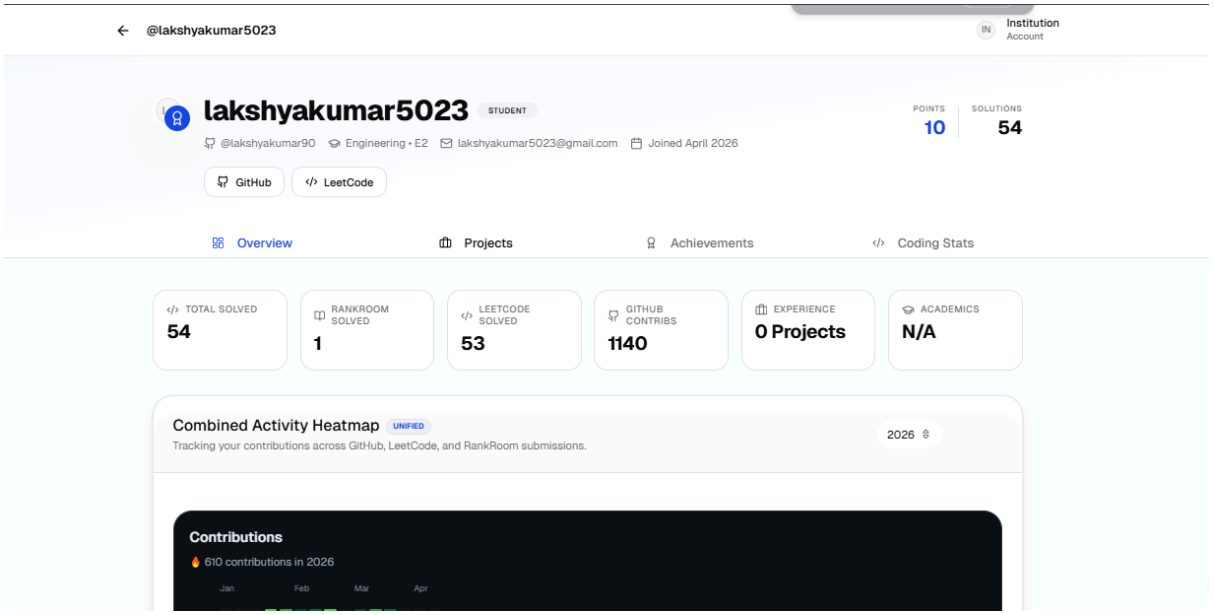


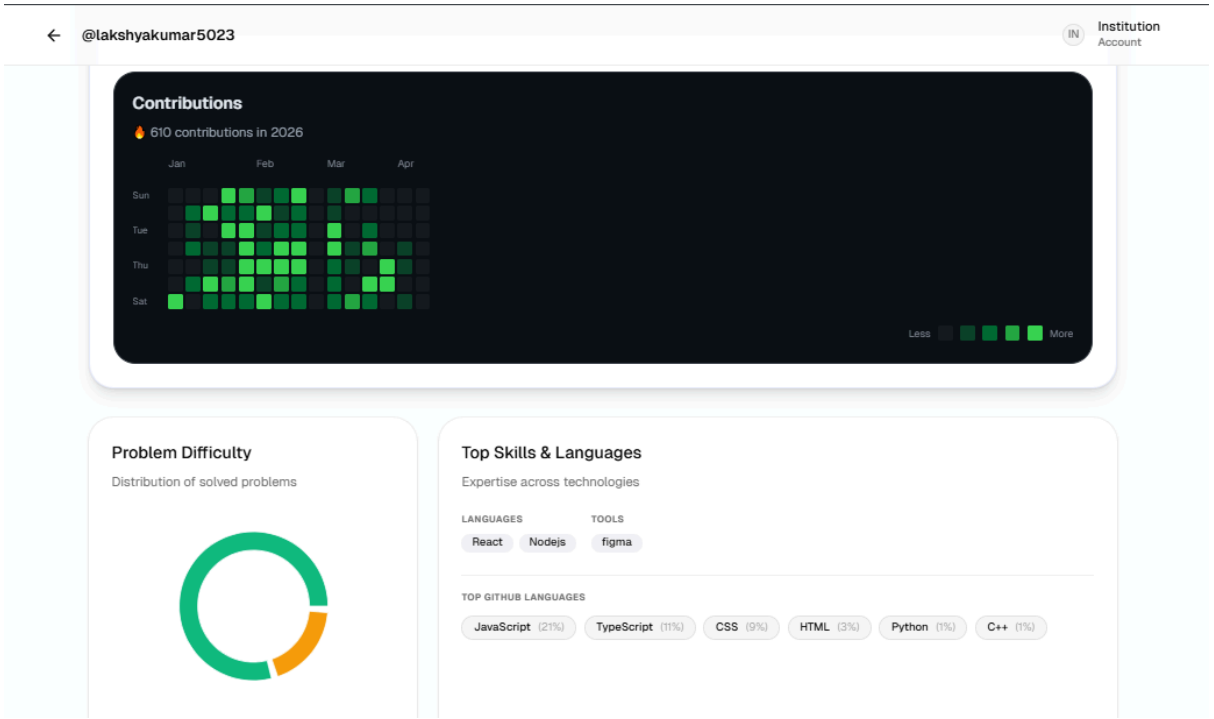
Problems Solving page





User Profile Page





REFERENCES

- Learning Management Systems: Concepts and Applications – Educational Technology Journal
- Role-Based Access Control (RBAC) Models – NIST Standard
- Modern Web Application Architecture – REST API Design Principles
- Cloud Computing Concepts – Distributed Systems and Scalability
- Gamification in Education – Research Papers on Student Engagement
- Official Documentation:
- React.js Documentation
- Node.js Documentation
- PostgreSQL Documentation
- REST API Design Guidelines
- Docker
- Supabase